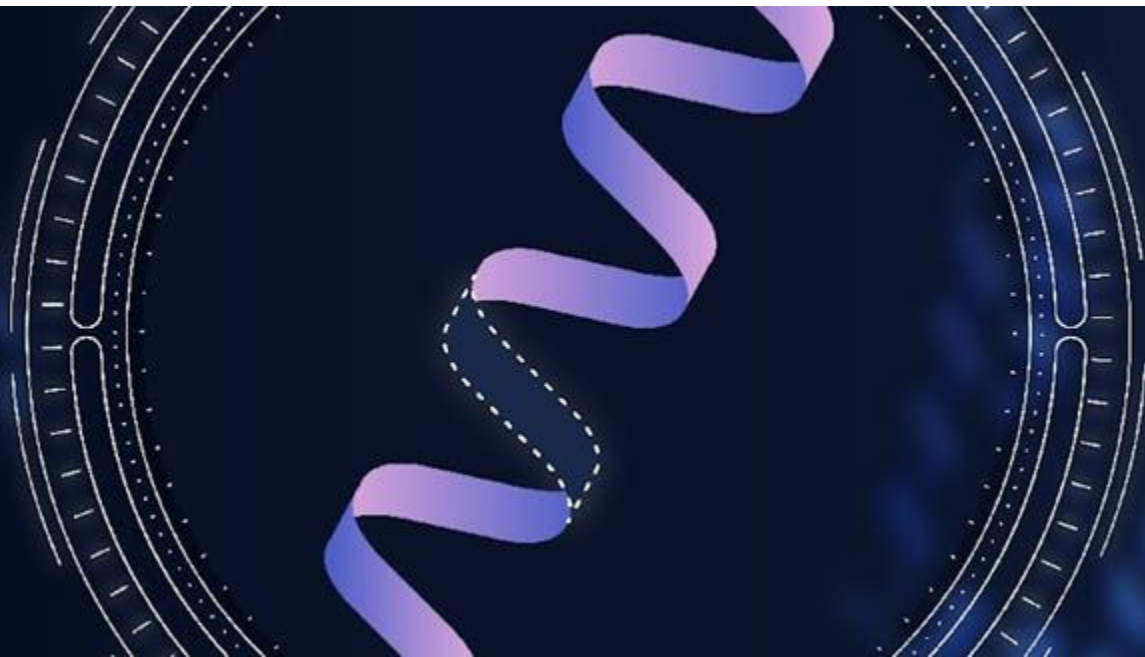


GraphNNs and GenAI in Drug Discovery & Design

Vladimir Gligorijević

PSI 2025 ML-SCI

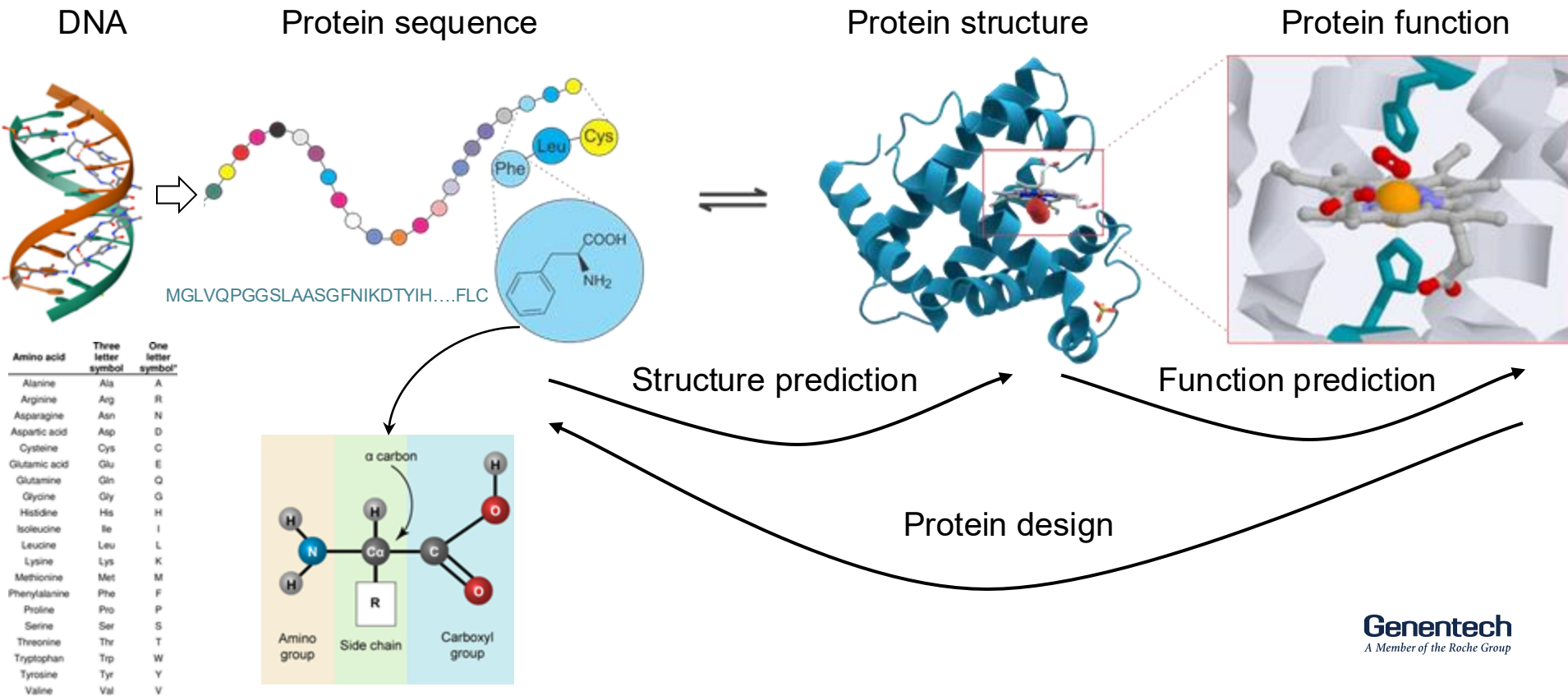
Petnica, Aug 8, 2025



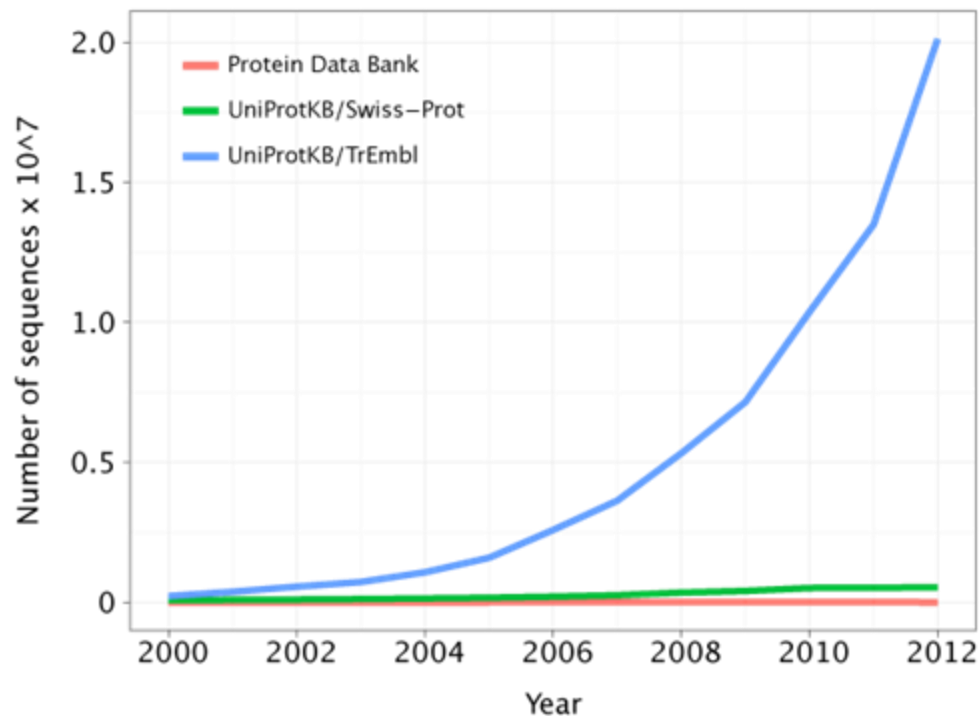
Outline

- Basic concepts in protein biology & biotech industry
- Molecule representation (sequences & graphs)
- Model architectures for modeling sequence and structure data: ConvNN, RNN, Transformers, GraphNN, etc.
- Practical work:
https://github.com/VGligorijevic/PSI-ML_2025-protein-folds
- Generative AI: diffusion models, multi-objective optimization, active learning, etc.

Central dogma in molecular biology



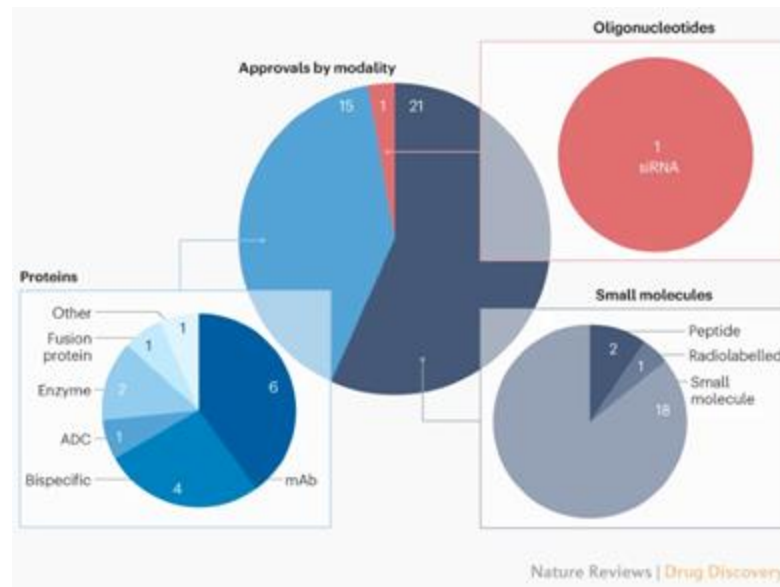
Publicly available datasets



Antibodies: our molecules of obsession



Antibodies: Y-shaped **proteins** belonging to the immunoglobulin superfamily



Abs were 20% of FDA approved drugs in 2022 (Mullard 2023)

Image credit: Antibody drug conjugate: the “biological missile” for targeted cancer therapy

Key diseases of application

Autoimmune diseases

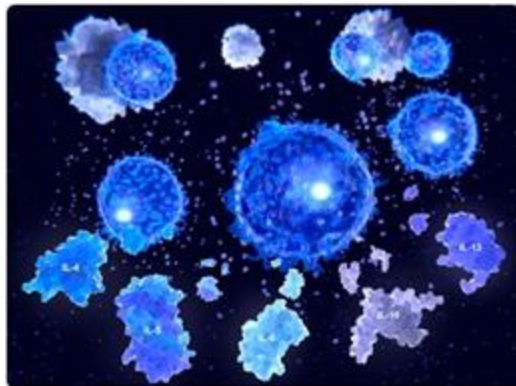
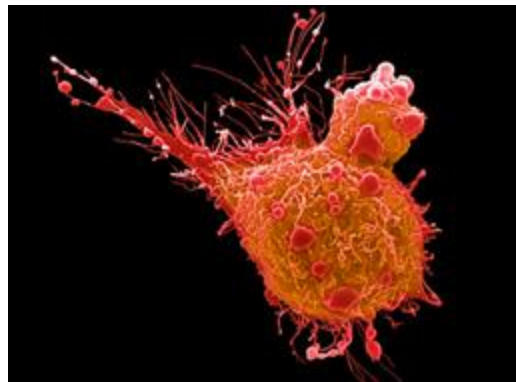


Image Credit: Juan Gaertner/Shutterstock.com

rheumatoid arthritis, Crohn's,
ulcerative colitis, kidney
transplants, asthma

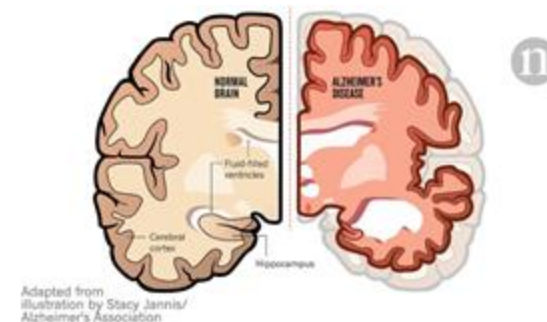
Cancer



Credit: Steve Gschmeissner/Science Photo Library

breast cancer, leukemia,
rectal/prostate cancer,
neuroblastoma lymphoma,
melanoma, ..

Alzheimer's Disease

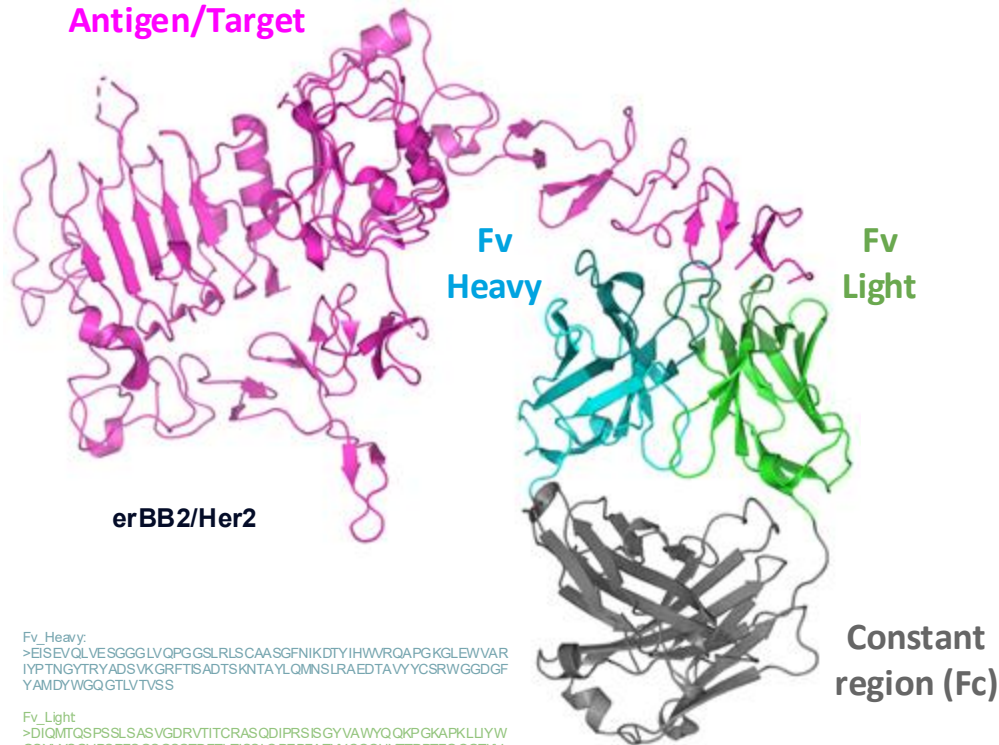


Adapted from
illustration by Stacy Jannits/
Alzheimer's Association

Lecanemab
27% improvement in
cognitive decline

Anatomy of an antibody

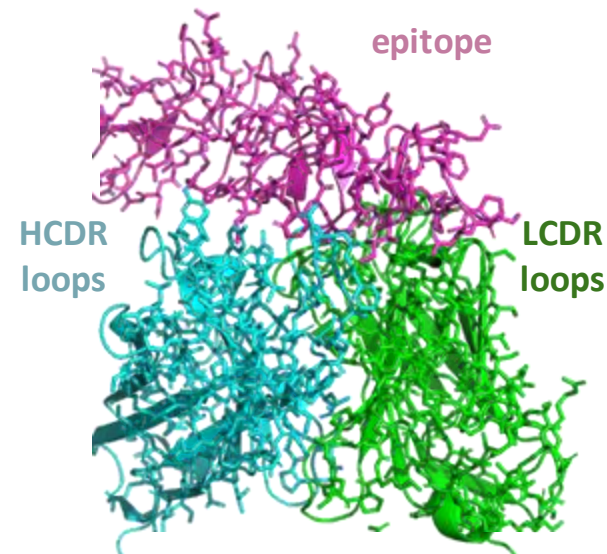
Antigen/Target



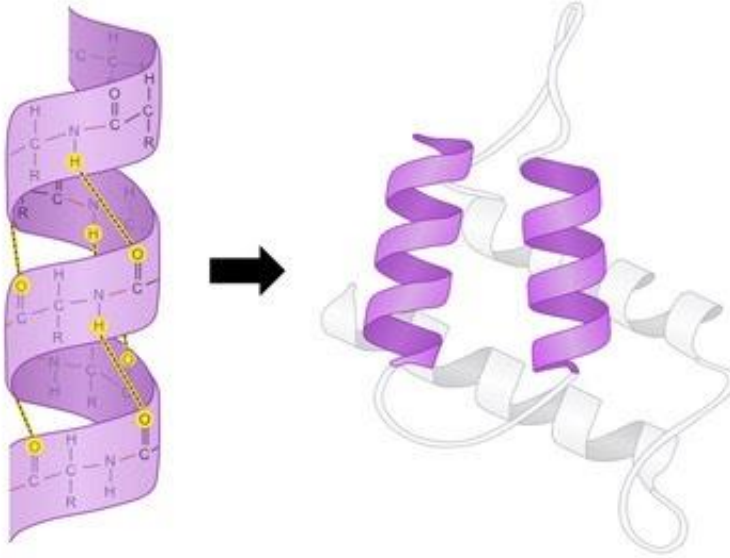
Fv_Heavy:
>EISEVQLVESGGGLVQPGGSLRLSCAASGFINIKDTYIHWRQAPGKLEWVAR
IYP TNGYTRYADSVKGRFTISADTSKNTAYLQMNSLR AEDTAVYYCSRWGGDGF
YAMDYWGQGTLLTVSS

Fv_Light
>DIQM TQSPSSLSASVGD RVTITCRASQDIPRSISGYVAWYQKPGKAPKLUYW
GSYL YSGVPSRFSGSGSDFTLTISSLQPEDFATYYCQQHYTTPPTFGQGTV
E

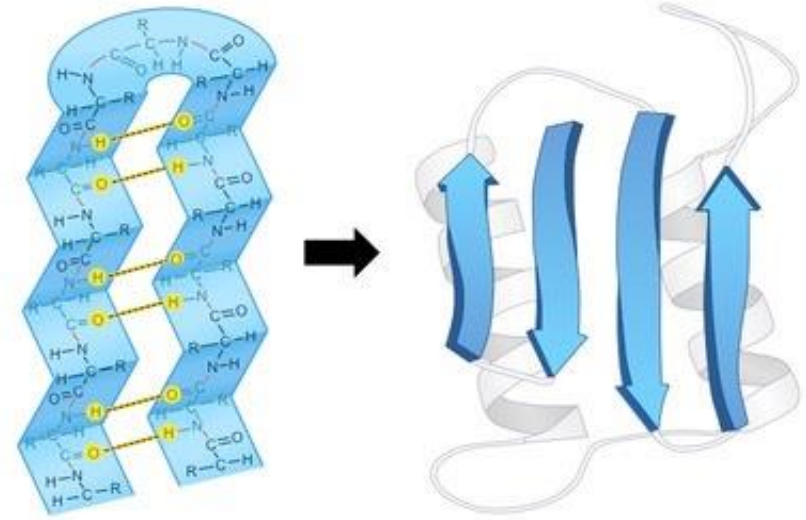
Trastuzumab (Herceptin®)



Protein secondary structure



Alpha-helices



Beta-pleated sheets

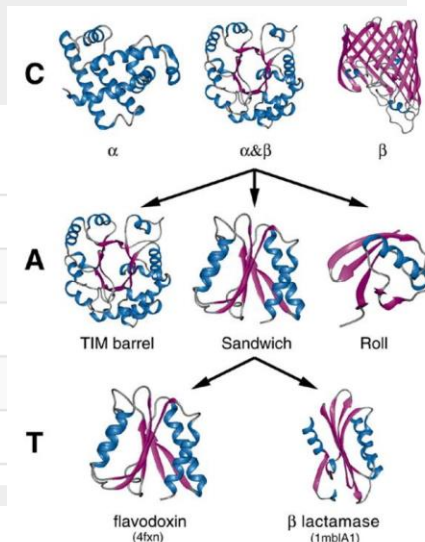
CATH classification of structures

- **Class** refers to the secondary structure content (e.g. mainly-alpha, mainly-beta, mixed alpha/beta or 'few secondary structures').
- **Architecture** refers to the general arrangement of the secondary structures irrespective of connectivity between them (e.g. alpha/beta sandwich).
- **Topology**, also known as the 'fold' level, takes into account the connectivity of secondary structures in the chain.
- **Homologous Superfamily** refers to domains that are believed to be related by a common ancestor.

Each level has a **CATH code** associated with it. Have a look at the following:

CATH Classification

Level	CATH Code	Description
	3	Alpha Beta
	3.40	3-Layer(aba) Sandwich
	3.40.50	Rossmann fold
	3.40.50.620	HUPs



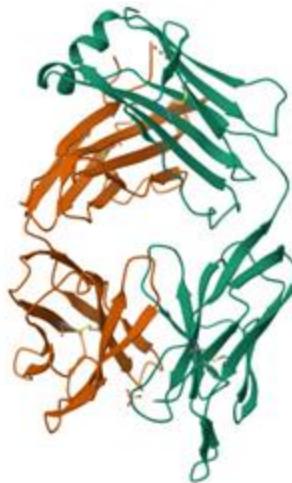
How do we represent proteins/antibodies in our ML algorithms ?

Sequence Features

- Amino Acid Sequence (e.g., 1-hot encoding)
- Chemical descriptors
- Evolutionary information (MSAs, numbering schemes; e.g., AHo)

Structural Features

- Voxelization (atom point cloud)
- Protein Graph (inter-residue distances/contact map + torsion angles)
- Secondary structure
- Protein surface



Learned Sequence Features

- ProtVec
- Autoencoders (AE)
- Language Models (ESM, UniRep, AntiBerty, etc.)

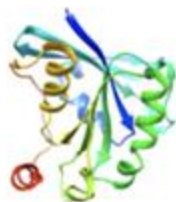
Learned Structural Features

- Surface fingerprints (dMASIF)
- Tokenizing the structure + Language Models (multi-modal LMs)

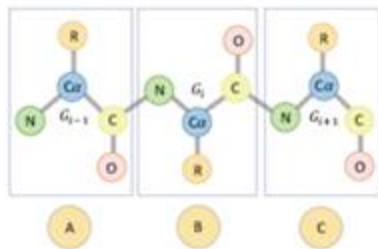
Protein featurization for ML models

```
AREALVPFG-- -KDAPLLIAV TVLTSMEASD LVD-LGMTLS
SREILEPYG-- -KERPLLIGV TVLTSMESAD LQG-IGILSA
VVNALQSITL- -KEKPLLIGV TILTSLDGSD LKT-LGIQEK
AKDGLIAGTPA GHSVPKLLAV TELTSLSDDV LRNEQNCRLP
```

Original AA sequence or MSA (1-hot encoding)



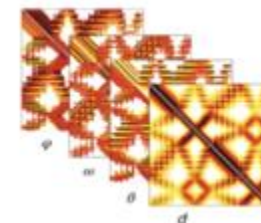
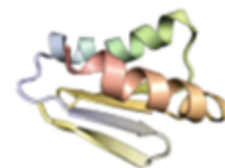
...ABC...



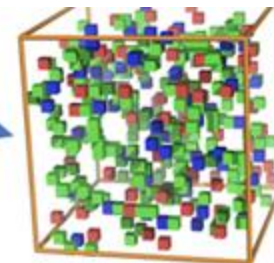
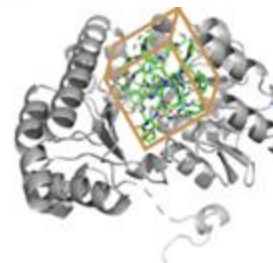
Backbone “frames” (used in AlphaFold, RosettaFold, etc.)



Protein graph (GNNs, EGNNs, etc.)



Distance and orientations (DeepAb, trRosetta, etc.)



Voxelized atom point cloud (3D CNNs, MAE, etc.)

1D Representations

Sequence

Physicochemical descriptors

2D Representations

Contact maps

Graphs

3D Representations

Surfaces

Point clouds

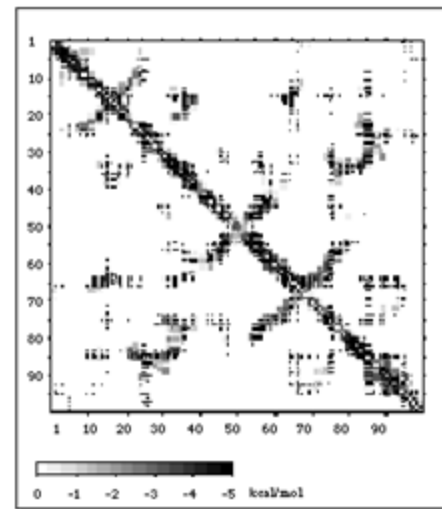
Geometric graphs

4D Representations?

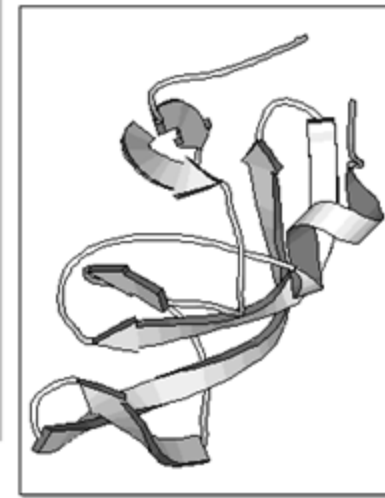
Structure + Dynamics!

P	PP	P	128	110
Q	QQQY		175	97
I	FFQYI		70	E 60
T	SSIVR		77	E 69
L	LLSTL		120	E 14
W	WWQED		238	E 81
Q	RRQAK		169	E 97
R	RRRPQ		200	62
P	PPPPP		24	48
L	VVTKF	E	71	E 59
V	VVLII	E	14	E 0
T	TTKEK	E	74	E 69
I	AALIV	E	0	E 0
K	HYKGF	E	90	E 73
I	IILVI		4	E 0
G	EENGG		46	41
G	GGGTG		62	53
Q	QQRKR		68	71
L	PPLWW	E	118	E 59
K	VVFKY	E	31	E 73
E	EESIK	E	124	E 95
A	VVGLG	E	1	E 0
L	LLILL	E	29	E 0
L	LLLVV	E	24	E 0
D	DDDDD		49	E 50
T	TTTTT		72	51
G	GGGGG		62	30
A	AAAAA		17	0
D	DDDDD		102	79
D	DDAKE		69	58
T	SSTTV		1	69
V	IIVIV	E	14	E 0
L	VVITL	E	0	E 0

1D



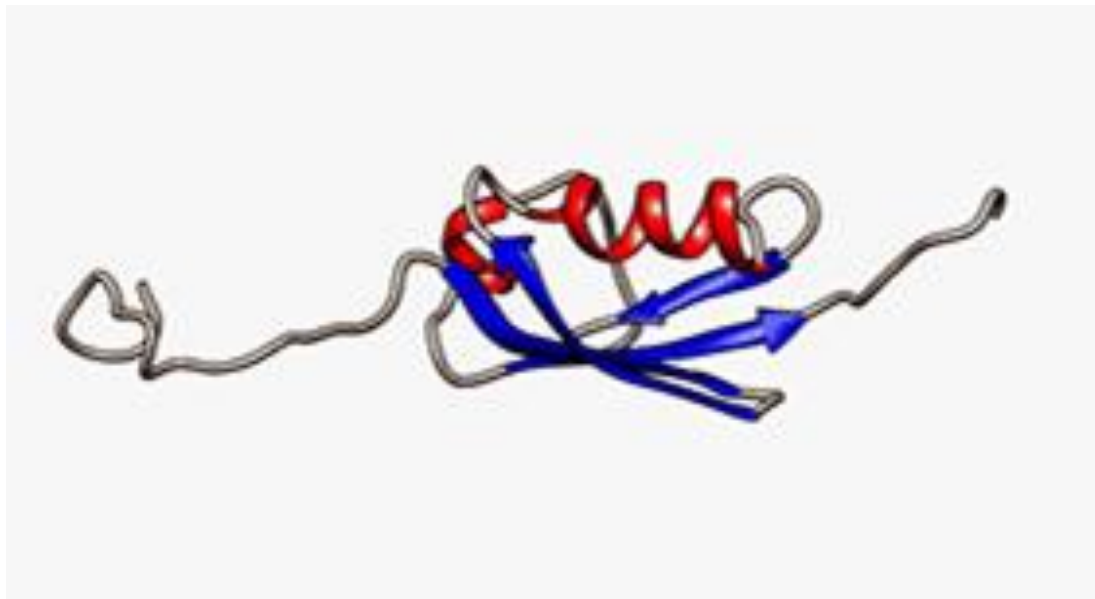
2D



3D

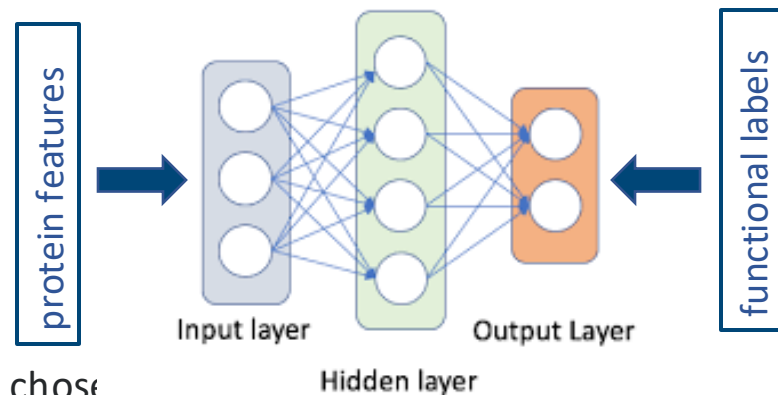
Protein dynamics

Proteins move(!) How can we incorporate this into ML models in a principled manner?



Deep Learning vs. Traditional ML

- Traditional ML methods (e.g., SVM, Logistic Regression, Random Forest, KNNs, etc.)
 - require feature engineering
 - offer more interpretable solutions
 - usually used in low-data regime
- Deep Neural Networks
 - high-level features
 - don't require feature engineering (depends on the choice)
 - require lots of data points



Protein representation

- Sequence representation (example: TRI27_HUMAN):

k-mers (k=3):

MASGSVAEC...SMETSP
MASGSVAEC...SMETSP
MASGSVAEC...SMETSP
MASGSVAEC...SMETSP
MASGSVAEC...SMETSP
MASGSVAEC...SMETSP
MASGSVAEC...SMETSP
MASGSVAEC...SMETSP
MASGSVAEC...SMETSP
MASGSVAEC...SMETSP
MASGSVAEC...SMETSP

MAS GTP MET ... ETS TSP

counts = [1, 0, 6, ..., 4, 2]

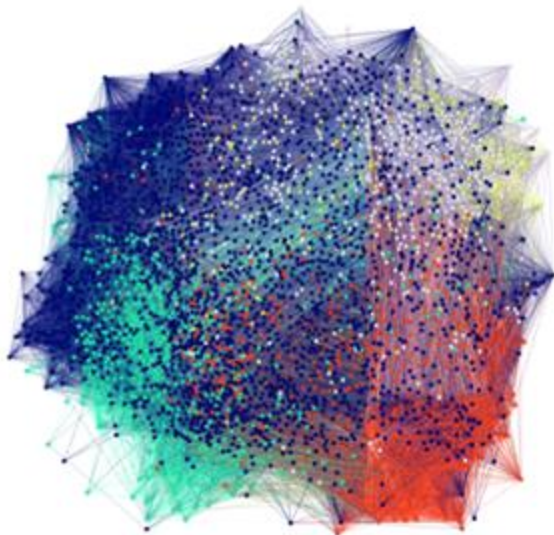
1-hot encoding:

M	0	0	0	1	0	...	0	0
A	1	0	0	0	0		0	0
S	0	1	0	0	0		0	0
G	0	0	0	0	0		0	0
S	0	1	0	0	0	...	0	0
V	0	0	0	0	0		0	0
A	1	0	0	0	0		0	0
E	0	0	1	0	0		0	0
C	0	0	0	0	1	...	0	0
⋮	⋮							⋮
S	0	1	0	0	0	...	0	0
M	0	0	0	1	0		0	0
E	0	0	1	0	0		0	0
T	0	0	0	0	0	...	1	0
S	0	1	0	0	0		0	0
P	0	0	0	0	0	...	0	1

MSA PSSM:

M	-1	-2	-1	4	-1	...	2	-1
A	-1	3	-4	-2	2		7	-1
S	-2	8	-2	-1	0		0	-2
G	-7	-3	-1	0	-12		2	-7
S	-3	-8	-2	9	-3	...	-1	-1
V	-4	-2	-7	4	-1		1	-1
A	-1	-2	-1	0	-9		8	-1
E	-2	-2	-1	-11	-1		-9	-1
C	-3	-2	-10	4	-1	...	7	-1
⋮	⋮							⋮
S	-7	-2	-5	4	5	...	2	-1
M	-3	-3	-7	9	-13		-3	-1
E	-1	-2	-1	4	-7		-5	-1
T	-7	-22	-1	7	-1	...	3	-1
S	-12	-2	-2	2	-14		-6	-1
P	-16	-25	0	-17	-1	...	7	-1

Features: broad view of biology requires diverse features



Networks

```

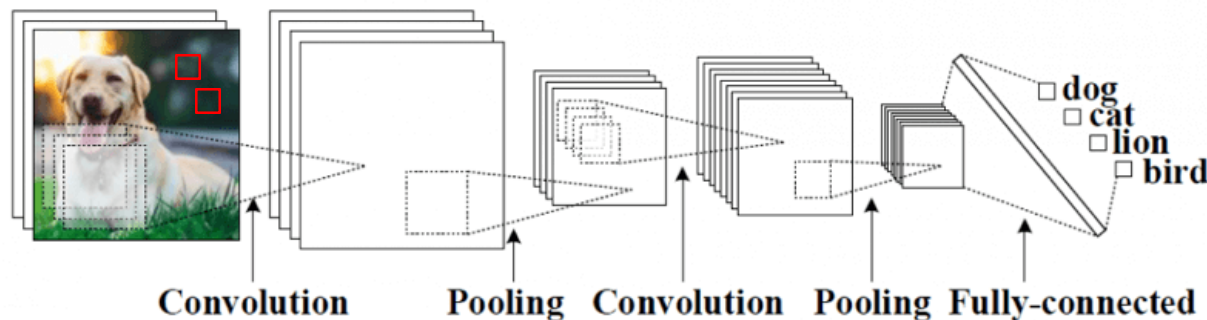
FRHVLVDALLKRRH.SVSAAR.ELDL...POPTASHGLARLRK.ALQDF
PRLLRLFDALFTTQ.SVTKAAE.RLQ...SOPTVSIWLRARLQ.ELDDF
PKLLQLFDVLYQCR.SVTRAAE.QLQ...SOPTISIWLRARLRE.QLNDF
FMHLLTVQALMRK.SVSLAAE.MLDH...POPTLSRHLKOLRE.HFGNQ
N....VFDALYRHQ.SAGKAAN.ALSM...POPTLSRWLAOLRT.HFDDF
LNLLVSLDTLLAEN.SVERAAE.RLGL...SNSAMSRILSRIRE.IFGDF
LNLLFALDVLLAEG.SVARAAR.RLRL...SPSAMSRILSRIRE.ATGDF
LNLLVTNLALLAEG.SVACAAE.RLGL...SPSAMSRILSRIRE.TMNDP
MNLLVALDALLDEG.SVVGAAQ.RMNL...SPAAMSRILSRIRE.ALQDF
LNMLVALNALIEER.SVTCATR.RLHL...SSPMSRPLARLRE.TLDDF
LNLLAALDALLEEG.SVACAAA.RLHV...TAPAMSRILSRIRE.TTGDF
LNLLVCLKVLIEEL.SVTRAAA.RLCL...SQSAVSKSLAKLRE.QFDDF
LNLLVCLKVLIEEL.NVTRTAH.RLCL...SQSAVSKALAKLQ.QFDDF
LNLLVILKVLIEEQ.SVTRAAE.RLHI...SQSALSKSLNRRLD.TLDDF
LNLLKVLAVMLEER.SVARCAE.RLHV...SPSAVSHALAKLQ.MFDDF
LNLLVALRVLIEES.SVSKAAV.RLNL...SQSAMSRVLGRRLD.LLQDF
LNLLKALDALLDER.NVTRAAV.RLAL...TQPAMSGMLTRRLD.SFDDF
LNLLMIFDAILAEG.NVTRAAE.RLAM...SQSAVSKGLAOLQ.AFGDF
LNLLPILALHDAE.SVSMQAQ.OLCH...SQPGVSTALAKLRT.AFGDF
LNLLVTLOVLLTEH.NVTRAAE.RLNF...SQPSVSVHLAKLRA.IFGDF
LNLLVSLDVLLIEC.NVTRAAQ.RLHV...SQPALSAOLGRRLH.LFDDF
LNLLVTLAVLLRER.SVSRAAA.CLHL...SQPAVSGALGRRLH.LFDDF
LNLLVVLDALEER.HVSRAAA.RLAM...SQPAVSHALGRRLH.LFDDF
LNLLRVLDALLRER.NVSRAAE.RLAL...SQPAVSNALNRRLD.LLQDF
LNLLRVFDMLLREQ.NVSRAAE.RLAL...TOPTVSNALNRRLD.OLQDF
    
```

Sequence



Structure

Convolution Neural Networks

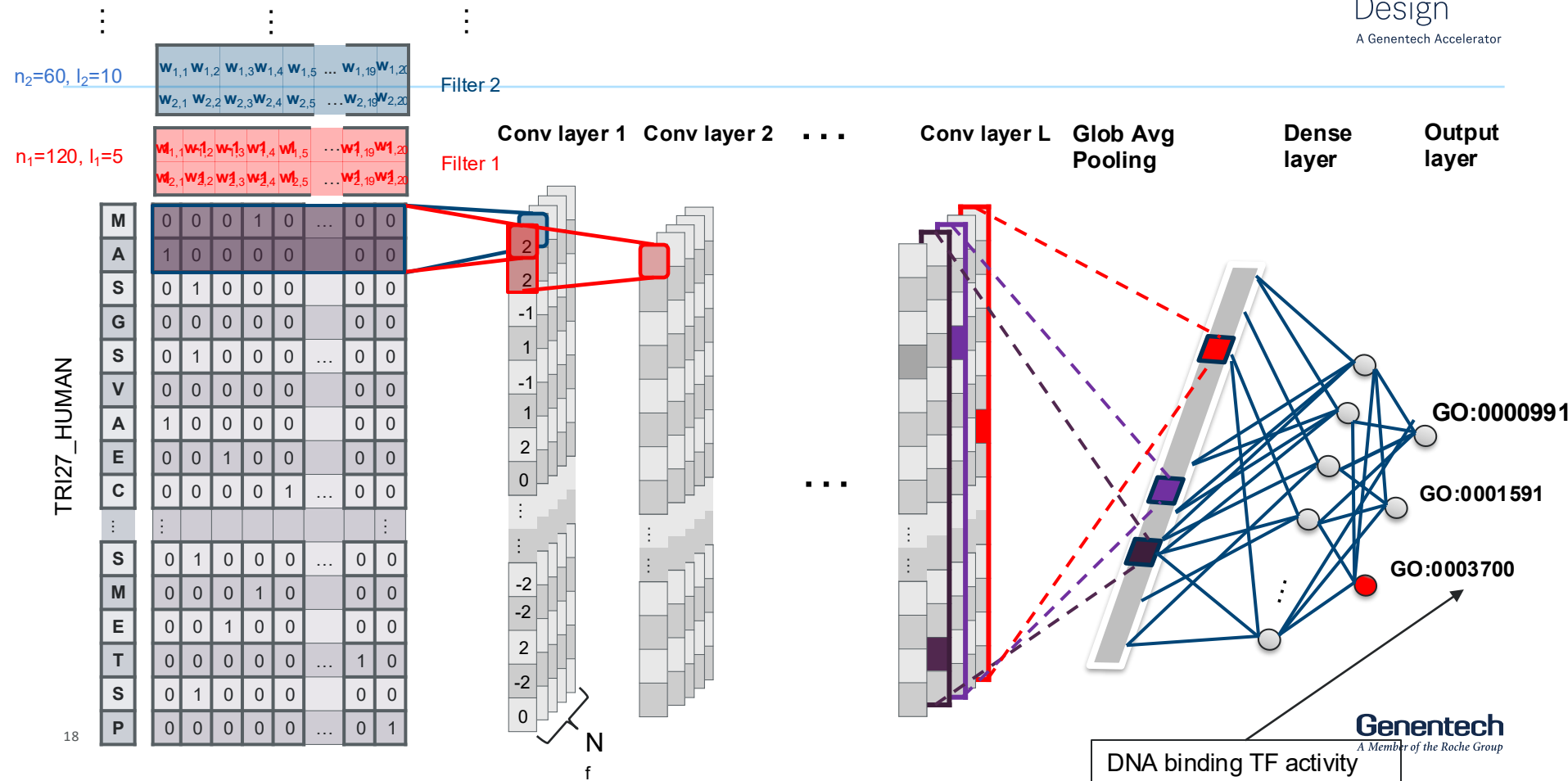


$$h_{ij}^k = \sigma((\mathbf{W}^k * x)_{ij} + \mathbf{b}_k)$$

where $\mathbf{W}^k$ are filters **learnable** through gradient descent

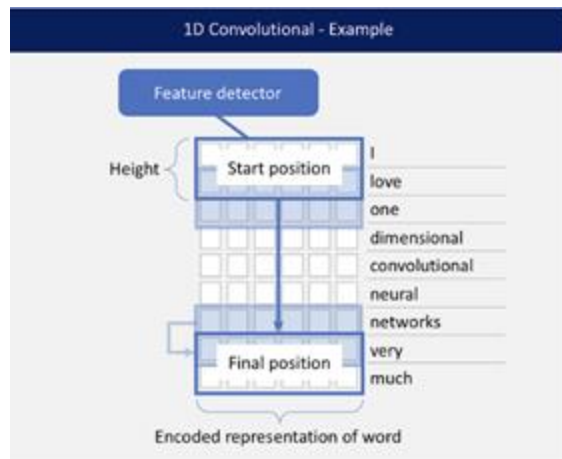
- Reduce number of parameters to be learned
- Localize and share weights (conv filters) → self-similarity
- Translation invariance
- Computationally efficient

CNN for sequence-to-function mapping



Architectural evolution

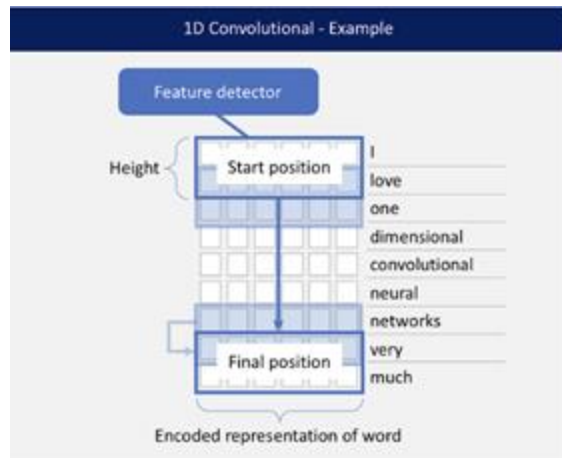
Convolutional Neural Networks



In this example for natural language processing, a sentence is made up of 9 words. Each word is a vector that represents a word as a low dimensional representation. The feature detector will always cover the whole word. The height determines how many words are considered when training the feature detector. In our example, the height is two. In this example the feature detector will iterate through the data 8 times.

Architectural evolution

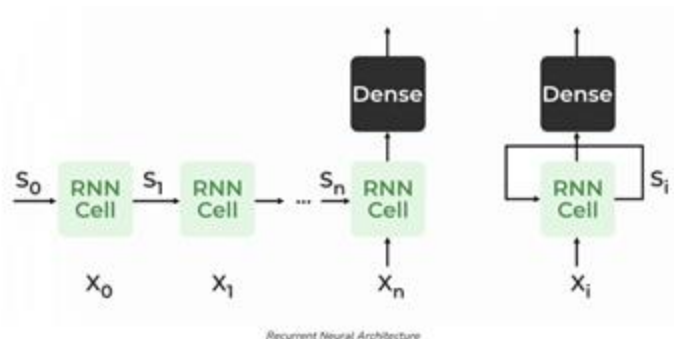
Convolutional Neural Networks



In this example for natural language processing, a sentence is made up of 9 words. Each word is a vector that represents a word as a low dimensional representation. The feature detector will always cover the whole word. The height determines how many words are considered when training the feature detector. In our example, the height is two. In this example the feature detector will iterate through the data 8 times.



Recurrent Neural Networks



<https://www.geeksforgeeks.org/introduction-to-recurrent-neural-network/>

GRU: On the Properties of Neural Machine
Translation: Encoder-Decoder
Approaches. Cho et al., 2014

Architectural evolution

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* †
illia.polosukhin@gmail.com

2017

120K citations

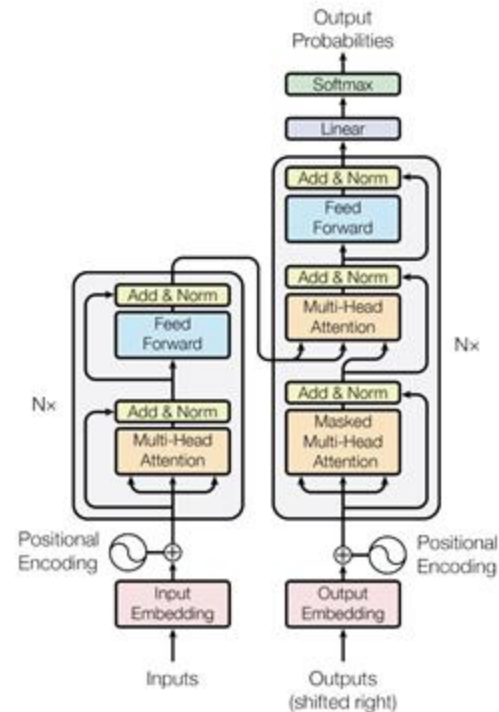
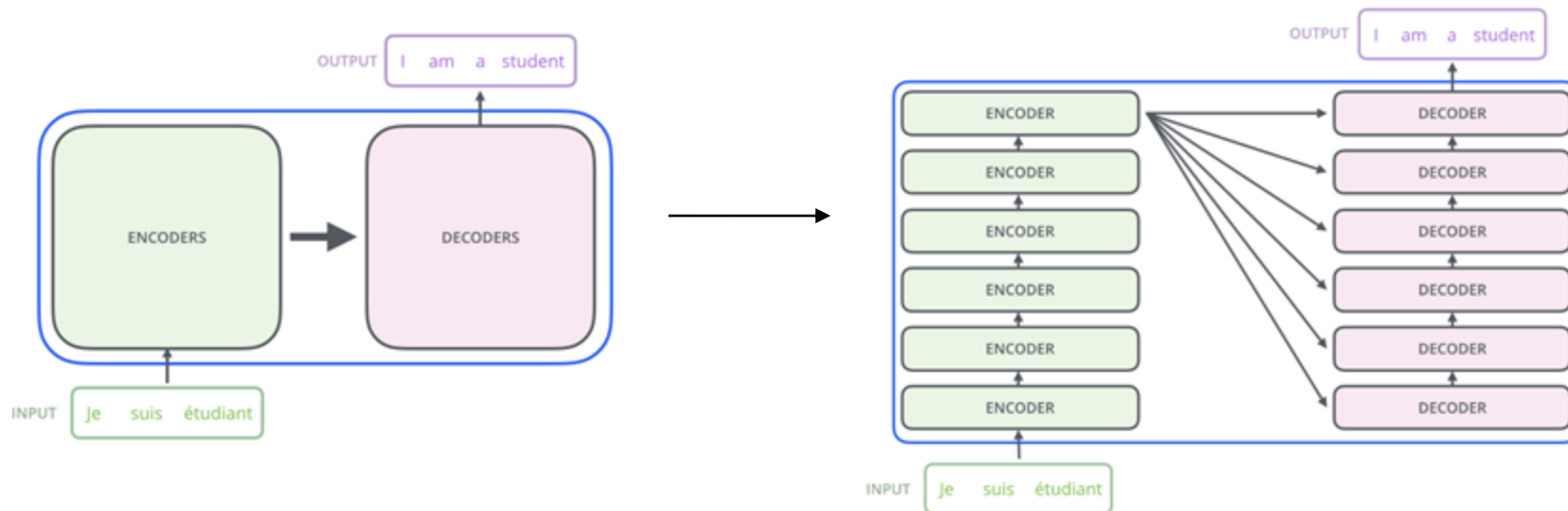


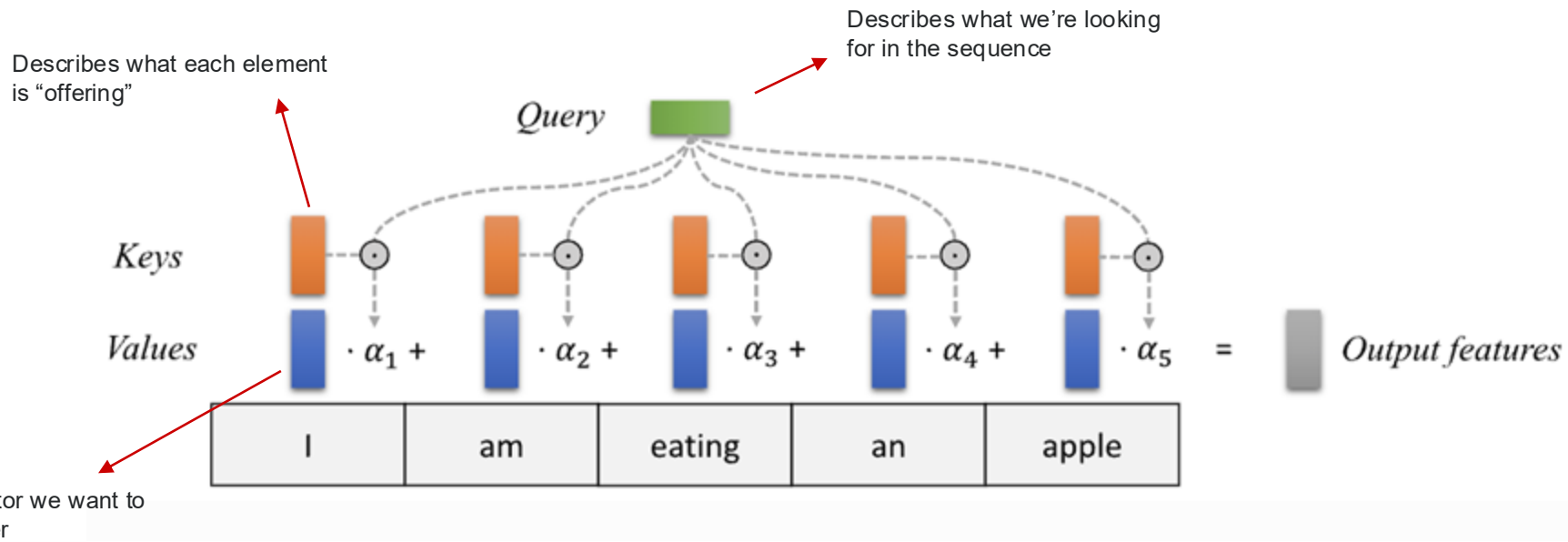
Figure 1: The Transformer - model architecture.

Transformers



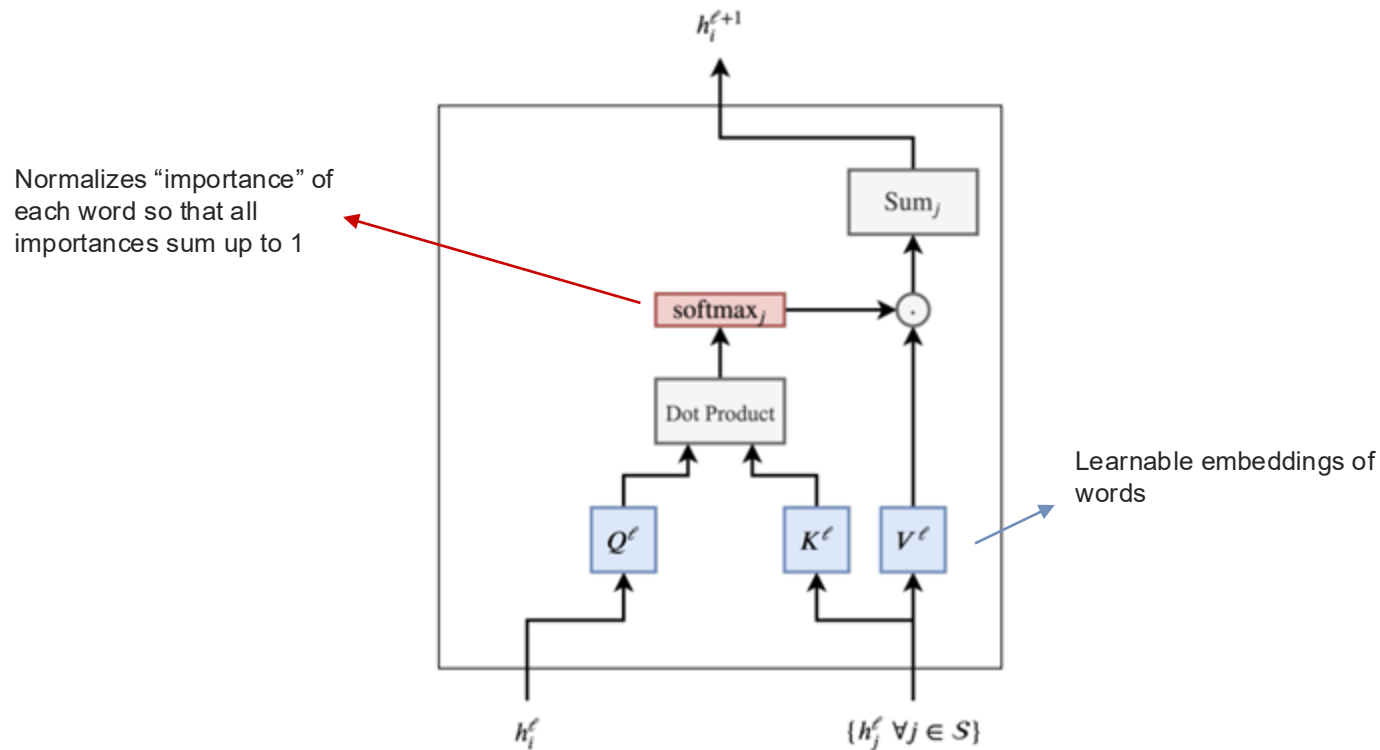
Transformers: attention mechanism

Attention



https://u vadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/tutorial6/Transformers_and_MHAttention.html

Transformers



ChatGPT 4o ▾



what are you?



I am an AI designed to assist you with your research and various tasks. How can I help you today?

how were you trained?



I was trained using a vast dataset that includes books, articles, websites, and other text sources. This dataset allows me to understand and generate human-like text. My training involved machine learning techniques, particularly deep learning models, to analyze patterns in the data and learn to predict and generate appropriate responses. This process was overseen by researchers to ensure accuracy and relevance. How can I assist you further?



Is this conversation helpful so far?



Message ChatGPT



ChatGPT can make mistakes. Check important info.

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina Toutanova
Google AI Language

{jacobdevlin, mingweichang, kentonl, kristout}@google.com

Research

RoBERTa: An optimized method for pretraining self-supervised NLP systems

7/29/2019

Protein language models (PLMs)

Evolutionary Scale Modeling (ESM) protein language models developed by researchers at Facebook AI Research (FAIR) and designed specifically for understanding and predicting the structure and function of proteins based on their amino acid sequences.

Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences

Alexander Rives^{a,b,1,2} ●, Joshua Meier^{a,1}, Tom Sercu^{a,1} ●, Siddharth Goyal^{a,1}, Zeming Lin^b, Jason Liu^a, Demi Guo^{c,3}, Myle Ott^a, C. Lawrence Zitnick^a, Jerry Ma^{d,e,3}, and Rob Fergus^b

^aFacebook AI Research, New York, NY 10003; ^bDepartment of Computer Science, New York University, New York, NY 10012; ^cHarvard University, Cambridge, MA 02138; ^dBooth School of Business, University of Chicago, Chicago, IL 60637; and ^eYale Law School, New Haven, CT 06511



Codebase: <https://github.com/facebookresearch/esm>

Protein language models (PLMs)

Evolutionary Scale Modeling (ESM) protein language models developed by researchers at Facebook AI Research (FAIR) and designed specifically for understanding and predicting the structure and function of proteins based on their amino acid sequences.

Meta Lays Off Team Behind Revolutionary Protein-folding Model

Former scientist and other sources claim that Meta is shifting its focus to monetizing AI models



Ben Wodecki, Jr. Editor
August 8, 2023

🕒 2 Min Read



Codebase: <https://github.com/facebookresearch/esm>

Protein language models (PLMs)

Evolutionary Scale Modeling (ESM) protein language models developed by researchers at Facebook AI Research (FAIR) and designed specifically for understanding and predicting the structure and function of proteins based on their amino acid sequences.



Protein-
AI models
EvolutionaryScale



Codebase: <https://github.com/facebookresearch/esm>

Protein language models (PLMs)

Main models you should use

Shorthand	<code>esm.pretrained.</code>	Dataset	Description
ESM-2	<code>esm2_t36_38_UR50()</code> <code>esm2_t48_158_UR50()</code>	UR50 (sample UR90)	SOTA general-purpose protein language model. Can be used to predict structure, function and other protein properties directly from individual sequences. Released with Lin et al. 2022 (Aug 2022 update).
ESMFold	<code>esmfold_v1()</code>	PDB + UR50	End-to-end single sequence 3D structure predictor (Nov 2022 update).
ESM-MSA-1b	<code>esm_msa1b_t12_100M_UR50S()</code>	UR50 + MSA	MSA Transformer language model. Can be used to extract embeddings from an MSA. Enables SOTA inference of structure. Released with Rao et al. 2021 (ICML'21 version, June 2021).
ESM-1v	<code>esm1v_t33_650M_UR90S_1()</code> ... <code>esm1v_t33_650M_UR90S_5()</code>	UR90	Language model specialized for prediction of variant effects. Enables SOTA zero-shot prediction of the functional effects of sequence variations. Same architecture as ESM-1b, but trained on UniRef90. Released with Meier et al. 2021 .
ESM-IF1	<code>esm_if1_gvp4_t16_142M_UR50()</code>	CATH + UR50	Inverse folding model. Can be used to design sequences for given structures, or to predict functional effects of sequence variation for given structures. Enables SOTA fixed backbone sequence design. Released with Hsu et al. 2022 .

For a complete list of available models, with details and release notes, see [Pre-trained Models](#).



Codebase: <https://github.com/facebookresearch/esm>

Protein language models (PLMs) - ESM2

RESEARCH

STRUCTURE PREDICTION

Evolutionary-scale prediction of atomic-level protein structure with a language model

Zeming Lin^{1,2†}, Halil Akin^{1†}, Roshan Rao^{1†}, Brian Hie^{1,3†}, Zhongkai Zhu¹, Wenting Lu¹, Nikita Smetanin¹, Robert Verkuil¹, Ori Kabeli¹, Yaniv Shmueli¹, Allan dos Santos Costa⁴, Maryam Fazel-Zarandi¹, Tom Sercu¹, Salvatore Candido¹, Alexander Rives^{1,2*}

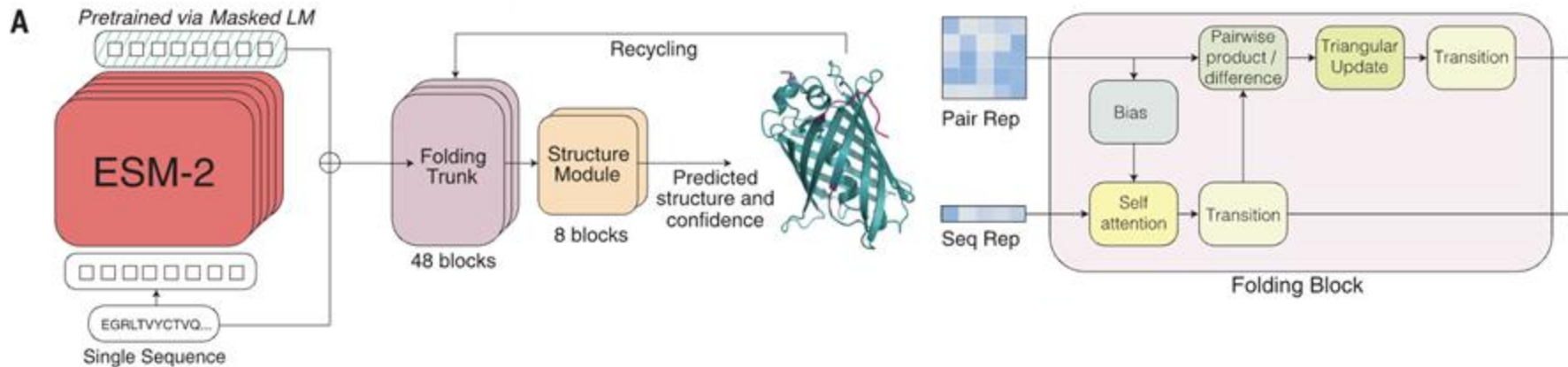
Direct inference of full atomic-level protein structure from primary sequence using a large language model.

15B parameters



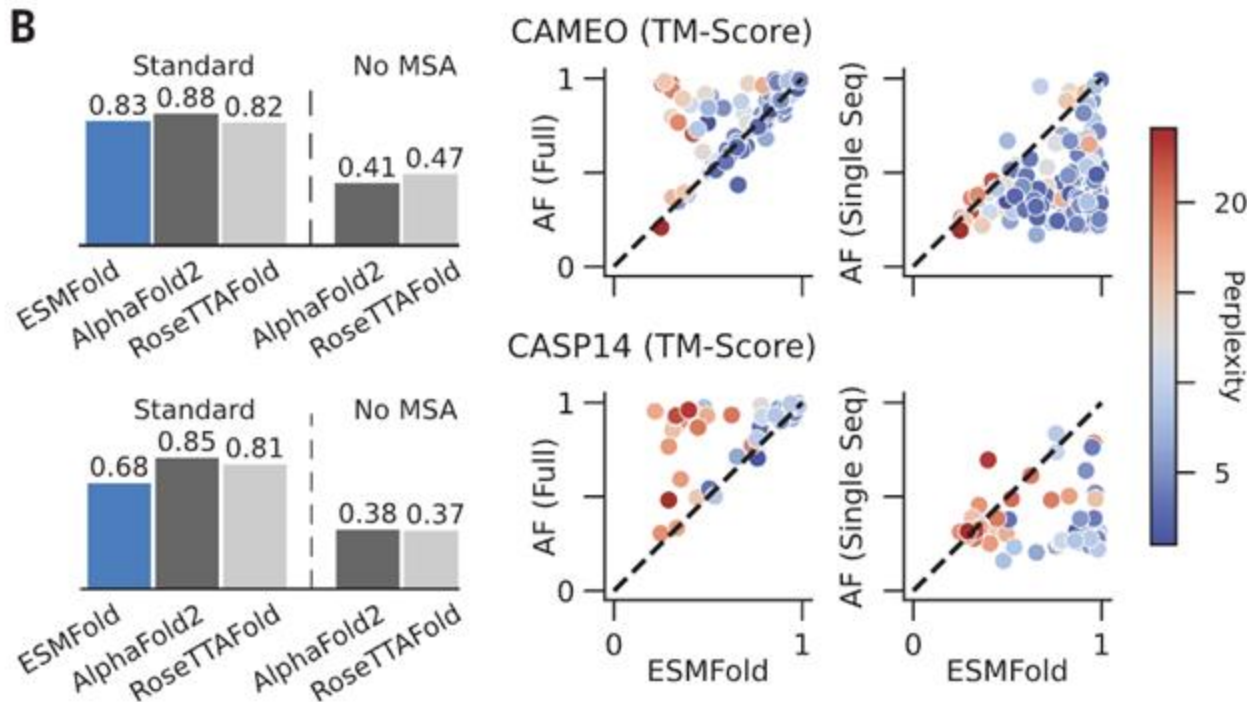
Codebase: <https://github.com/facebookresearch/esm>

Protein language models (PLMs) - ESM2



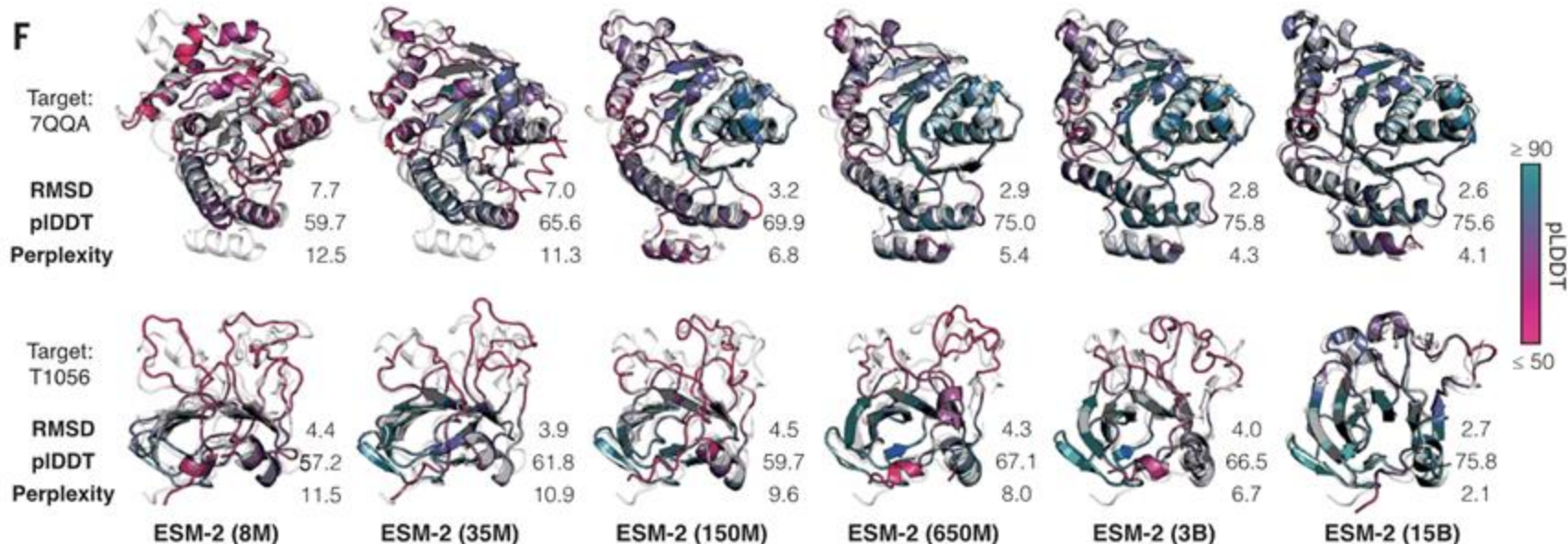
ESMFold model architecture. Arrows show the information flow in the network from the language model to the folding trunk to the structure module that outputs 3D coordinates and confidences. LM, language model.

Protein language models (PLMs) - ESM2



ESMFold produces accurate atomic resolution predictions, with similar accuracy to RoseTTAFold on CAMEO. When MSAs are ablated for AlphaFold2 and RoseTTAFold, performance of the models degrades. Scatter plots compare ESMFold (x axis) predictions with AlphaFold2 (y axis), colored by language model perplexity. Proteins with low perplexity score similarly to AlphaFold2. AF, AlphaFold2.

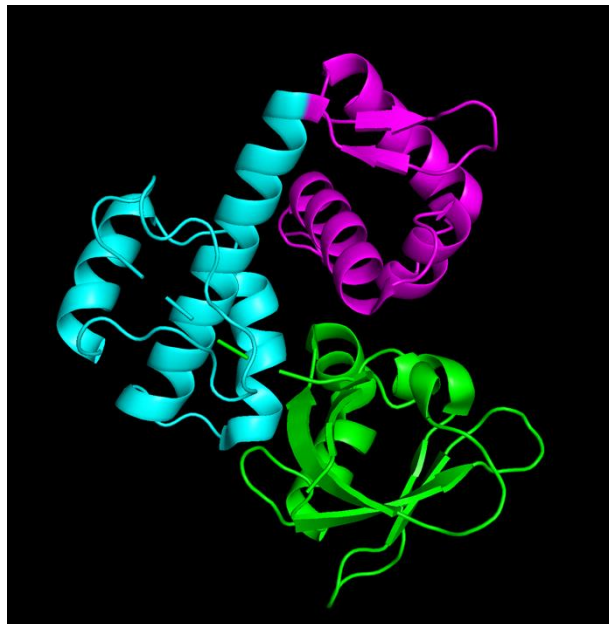
Protein language models (PLMs) - ESM2



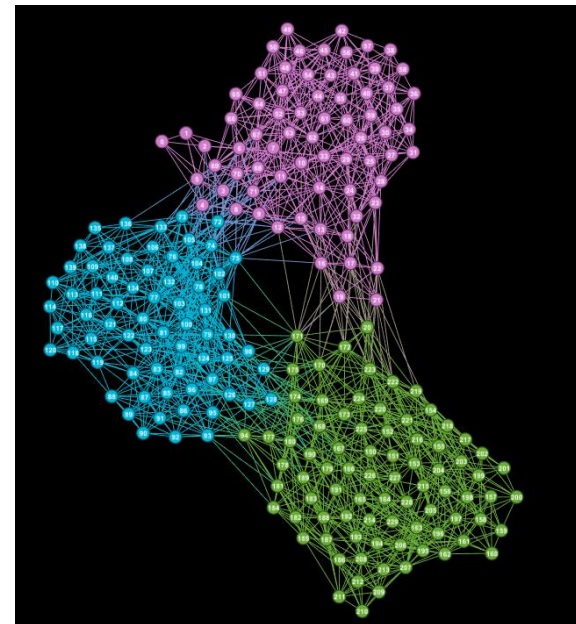
Structure predictions on CAMEO structure 7QQA and CASP target 1056 at all ESM-2 model scales, colored by pLDDT (pink, low; teal, high). For 7QQA, prediction accuracy improves at the 150-million-parameter threshold. For T1056, prediction accuracy improves at the 15-billion-parameter threshold.

GraphNN

Example PDB: 1U8R-B



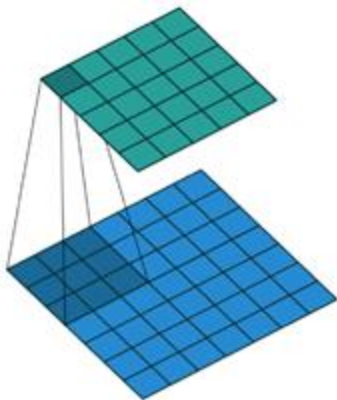
$C\alpha-C\alpha$
 $< 10\text{\AA}$



>>MNELVDTEMYLRTIYDLEEEGVTPLRARIAERLDQSGPTVSQTVSRMERDGLLRVAGDRHLELTKGRALAIAMVRKHRLAERLLVDVIGLPWEEVHAEACRWEHVMSEDEVRRLLVKVLNNPTSPFGNP
IPGLDELGVGPEPGADDANLVRTELPAAGSPVAVVVRQLTEHVQGDIDLITRLKDAVVPNARVTVETTPGGGVTVIPGHENVTLPEMAHAVKVEKV

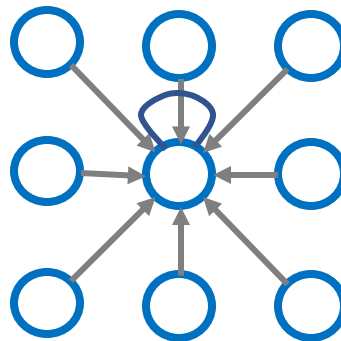
Geometric deep learning: CNN for graphs

CNN

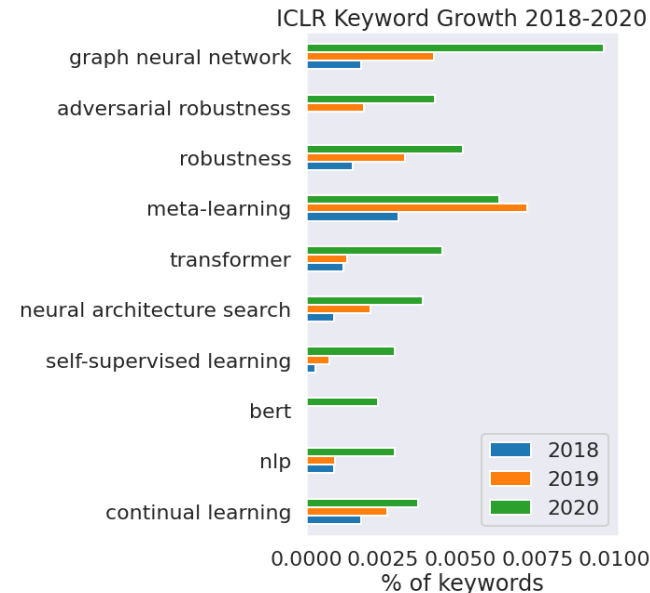


grid-like data structure

GraphNN



graph-like data structure

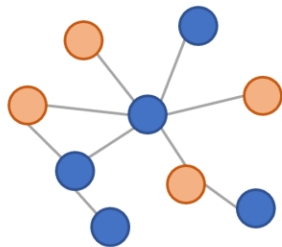


GraphNNs: types and applications

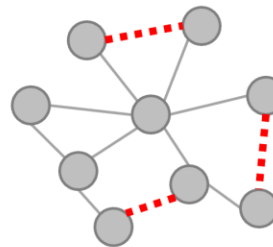
Types of GNNs:

- Spectral formulation
- Spatial formulation

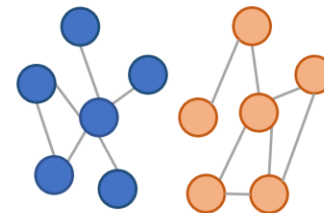
Node Classification



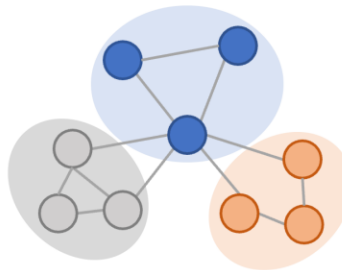
Link Prediction



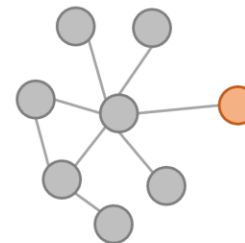
Graph Classification



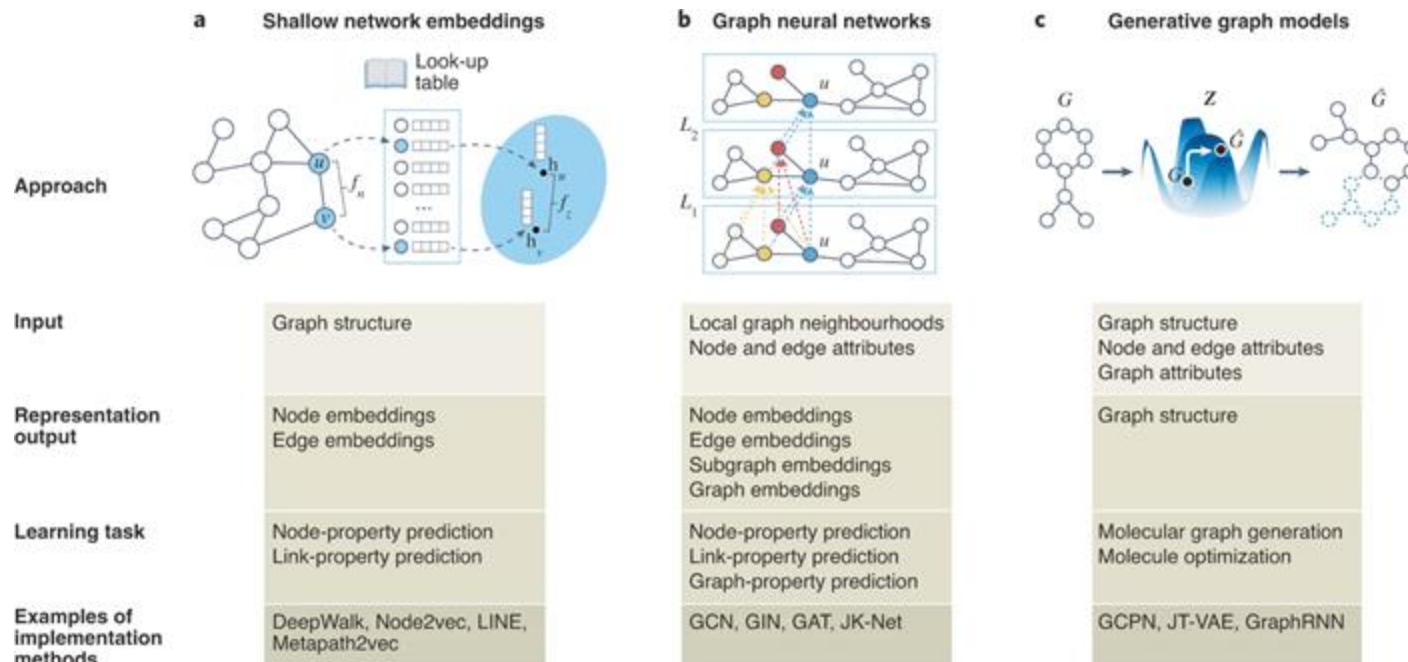
Community Detection



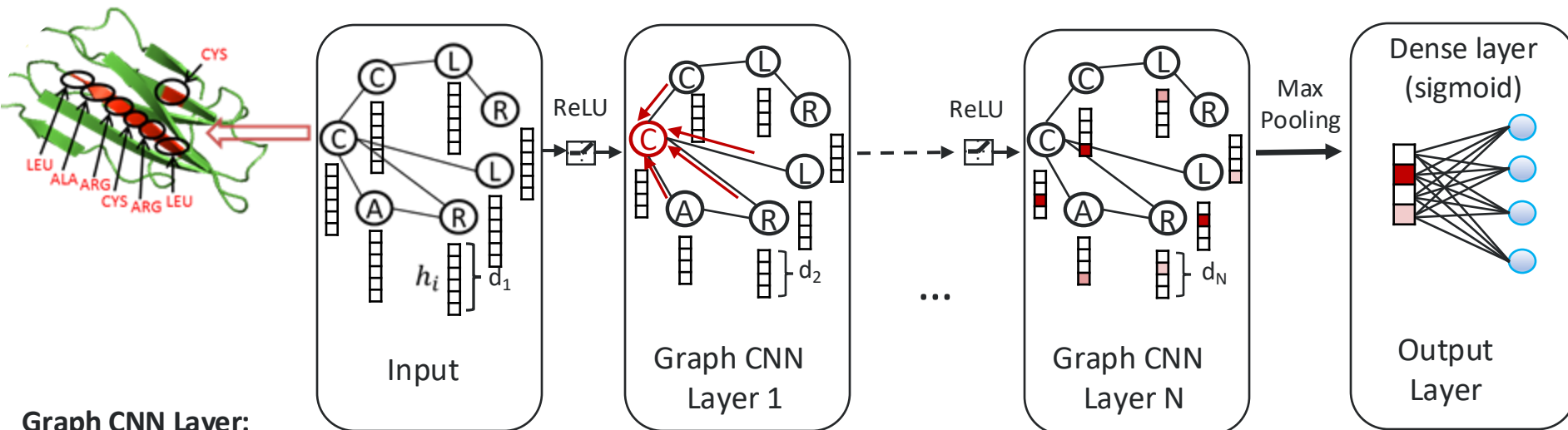
Anomaly Detection



History of the graph-based methods



An example of a GraphNN...



Graph CNN Layer:

$$H^{(l+1)} = \sigma[\hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}} H^{(l)} W^{(l)}]$$

Output to next layer / result	3. Nonlinearity	1. Normalize graph structure	2. Multiply node properties and weights
-------------------------------	-----------------	------------------------------	---

A – adjacency matrix representing contact map

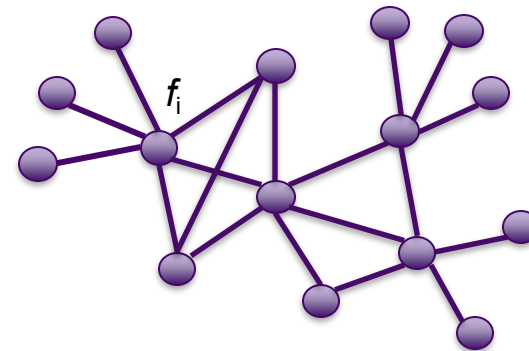
D – diagonal degree matrix

H⁽⁰⁾ – input feature matrix (**H⁽⁰⁾**=**I**, 1-hot encoding if features not available)

W^(l) – weight matrix for layer l

Brief intro to Graph Theory

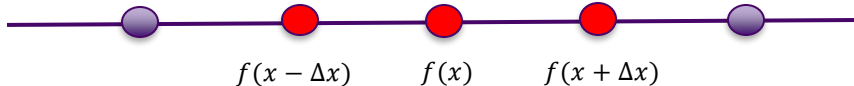
- Undirected graph: $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, with **vertices** $\mathcal{V} = \{1, \dots, n\}$ and **edges** $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$
- Represented by **adjacency matrix**, $\mathbf{A} \in \mathbb{R}^{n \times n}$, with $a_{ij} = 1$ if $(i, j) \in \mathcal{E}$ and $a_{ij} = 0$, otherwise
- Functions over graph vertices: $L^2(\mathcal{V}) = \{f : \mathcal{V} \rightarrow \mathbb{R}\}$ represented as vectors: $\mathbf{f} = (f_1, \dots, f_n) \in \mathbb{R}^n$
- Inner product operation: $\langle \mathbf{f}, \mathbf{g} \rangle_{L^2(V)} = \sum_{i \in \mathcal{V}} f_i g_i = \mathbf{f}^T \mathbf{g}$



Laplace operator

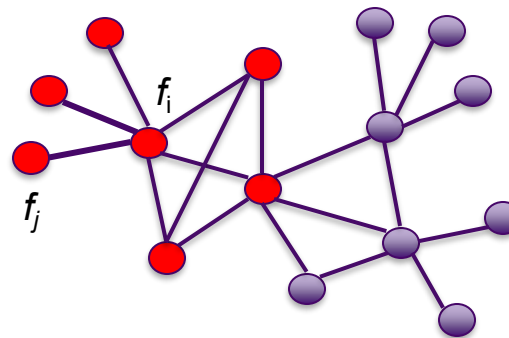
- Euclidean

- $\Delta : L^2 \rightarrow L^2$
- In $\mathbb{R}$: $\Delta f = \lim_{\delta x \rightarrow 0} \frac{f(x+\delta x) - 2f(x) + f(x-\delta x)}{(\delta x)^2}$
- Discrete Laplace operator:
 $\Delta f \propto [f(x) - f(x + \Delta x)] + [f(x) - f(x - \Delta x)]$



- Non-Euclidean

- $\Delta : L^2(\mathcal{V}) \rightarrow L^2(\mathcal{V})$
- On graphs: $(\Delta f)_i = \sum_{j:(i,j) \in \mathcal{E}} a_{ij}(f_i - f_j)$
- Matrix representation:
 $\Delta = \mathbf{D} - \mathbf{A}$, where $\mathbf{D} = \text{diag}(\sum_{j \neq i} a_{ij})$



Spectrum of the Laplacian

• Euclidean

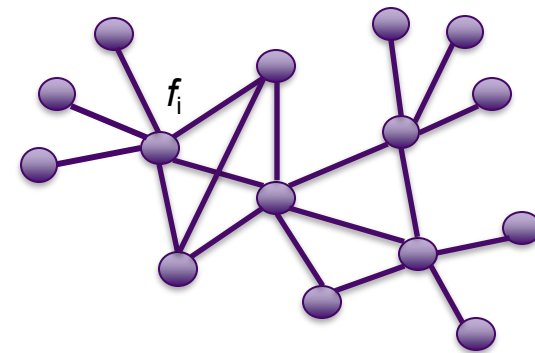
- Eigendecomposition: $-\Delta e^{2\pi i \xi x} = (2\pi \xi)^2 e^{-2\pi i \xi x}$
- Fourier transform: $\hat{f}(\xi) = \langle f, e^{2\pi i \xi x} \rangle = \int_{\mathbb{R}} f(x) e^{-2\pi i \xi x} dx$
- Inverse transform: $f(x) = \langle \hat{f}, e^{-2\pi i \xi x} \rangle = \int_{\mathbb{R}} \hat{f}(\xi) e^{2\pi i \xi x} d\xi$
- Fourier basis = Laplacian eigenfunctions

• Non-Euclidean

- Normalized Laplacian: $\hat{\Delta} = \mathbf{D}^{-\frac{1}{2}} \Delta \mathbf{D}^{-\frac{1}{2}} = \mathbf{I}_n - \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}$
- Eigendecomposition: $\hat{\Delta} \phi_i = \lambda_i \phi_i$;
 - Non-negative **eigenvalues**: $0 \leq \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n \leq 2$;
Matrix form: $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$
 - Corresponding orthogonal **eigenvectors**: $\langle \phi_i, \phi_j \rangle = \delta_{ij}$; Matrix form: $\mathbf{\Phi} = (\phi_1, \dots, \phi_n)$
- Eigendecomposition in matrix form: $\hat{\Delta} = \mathbf{\Phi} \mathbf{\Lambda} \mathbf{\Phi}^T$
- Fourier transform: $\hat{f}(\lambda_l) = \langle f, \phi_l \rangle = \sum_{i=1}^n f_i \phi_{li}^*$;
Matrix-vector form: $\hat{\mathbf{f}} = \mathbf{\Phi}^T \mathbf{f}$
- Inverse transform: $f_i = \langle f, \phi_i^* \rangle = \sum_{l=1}^n \hat{f}(\lambda_l) \phi_{li}$;
Matrix-vector form: $\mathbf{f} = \mathbf{\Phi} \hat{\mathbf{f}}$

Convolution operation

- Classically, for two signals $f, g \in L^2(\mathbb{R})$ the convolution is defined as: $f * g(x) = \int_{\mathbb{R}} f(u)g(x - u)du$
- No clear analogue for graphs (in spatial domain) $\implies$ compute convolution in the Fourier domain
- Convolution theorem: $\widehat{(f * g)}(\xi) = \hat{f}(\xi) \circ \hat{g}(\xi) \implies$ take inverse
- Convolution of signals on a graph (in matrix-vector notation): $\mathbf{f} * \mathbf{w} = \mathcal{F}^{-1}\{\hat{\mathbf{f}} \circ \hat{\mathbf{w}}\} = \Phi(\hat{\mathbf{w}} \circ \Phi^T \mathbf{f})$
- $\mathbf{f} * \mathbf{w} = \Phi \text{diag}(\hat{w}_1, \dots, \hat{w}_n) \Phi^T \mathbf{f}$



Filter coefficients depend on Laplacian eigenvectors!

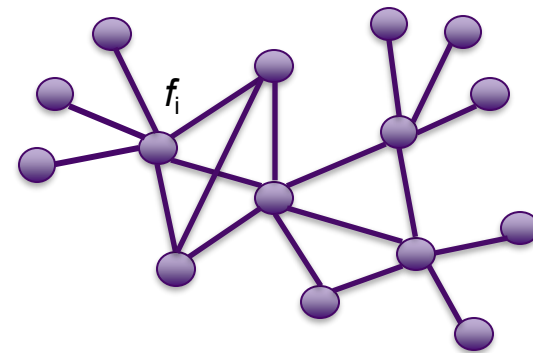
Spectral GraphConv layer

Convolution in spectral domain:

$$\mathbf{h} = \Phi \mathbf{W} \Phi^T \mathbf{f}$$

where $\mathbf{W}$ is $n \times n$ diagonal (learnable) matrix of spectral filter coefficients

- **Filters are basis-dependent $\rightarrow$ does not generalize across graphs** 😞
- **Learning complexity $O(n)$** 😞
- **Computational complexity $O(n^2)$** 😞
- **Filters are not localized in space** 😞



GraphConv with polynomial filters

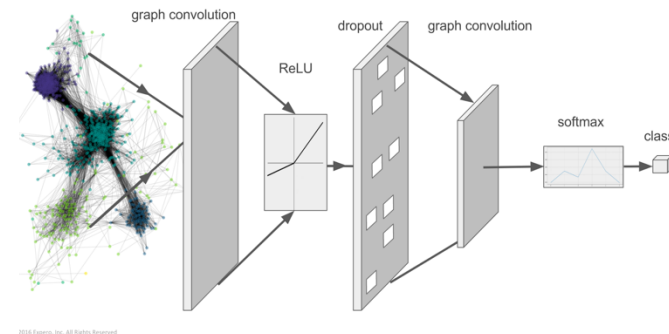
- Parametrize the filter using a smooth function:

$$\hat{g}_{\mathbf{w}}(\lambda) = \sum_{k=1}^K w_k \lambda^k$$
- Spectral filters represented by K^{th} order polynomials,

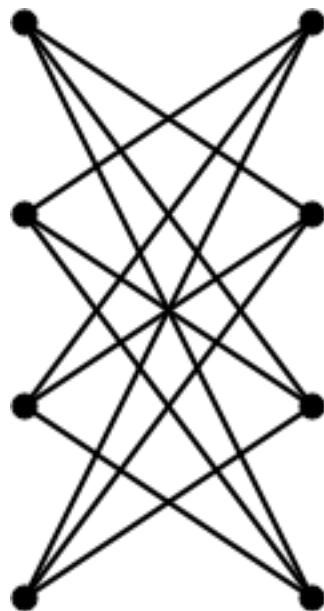
$$\hat{g}_{\mathbf{w}}(\Lambda) = \sum_{k=0}^K w_k \Lambda^k$$
, are **exactly K -localized**, where
 $\mathbf{w} = (w_1, \dots, w_K)$ is the vector of filter parameters
- Application of the filter: $\mathbf{h} = \Phi \hat{g}_{\mathbf{w}}(\Lambda) \Phi^T \mathbf{f} = \hat{g}_{\mathbf{w}}(\hat{\Delta}) \mathbf{f}$
- Number of parameters K (fixed) is independent of graph size n (same learning complexity as regular CNNs)
- No explicit computation of Φ , Φ^T because $\mathbf{h} = \hat{g}_w(\hat{\Delta}) \mathbf{f} \implies O(K|\mathcal{E}|)$ computational complexity

Graph Convolutional Networks

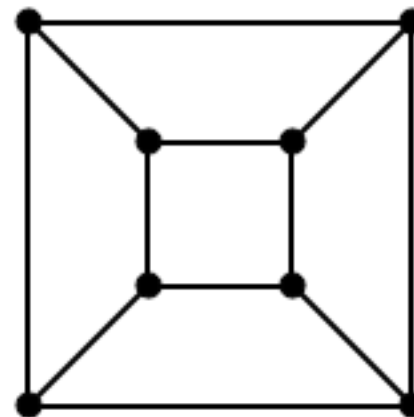
- First-order approximation ($K = 1$) of spectral graph convolution
- $\mathbf{h} = g_w(\hat{\Delta})\mathbf{x} = \sum_{k=0}^1 w_k \hat{\Delta}^k \mathbf{x} = w(\mathbf{I}_n + \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}) \mathbf{x}$, for signal $\mathbf{x} \in \mathbb{R}^n$
- Renormalization trick (numerical stability):
 $\mathbf{I}_n + \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}} \rightarrow \hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-\frac{1}{2}}$, with $\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}_N$ and $\hat{\mathbf{D}}_{ii} = \sum_j \hat{\mathbf{A}}_{ij}$
- Generalization to signals with C input channels, $\mathbf{X} \in \mathbb{R}^{N \times C}$, and F filters: $\mathbf{H} = \hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{X} \mathbf{W}$
 where $\mathbf{W} \in \mathbb{R}^{C \times F}$ is a matrix of filter parameters, and $\mathbf{H} \in \mathbb{R}^{N \times F}$ is the output activation matrix



Graph Isomorphism

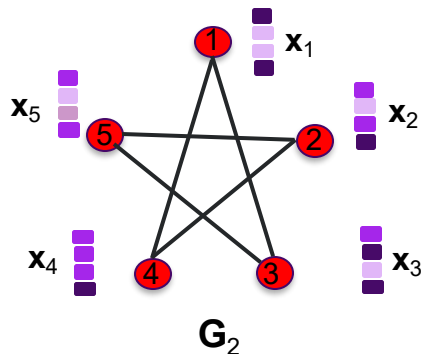
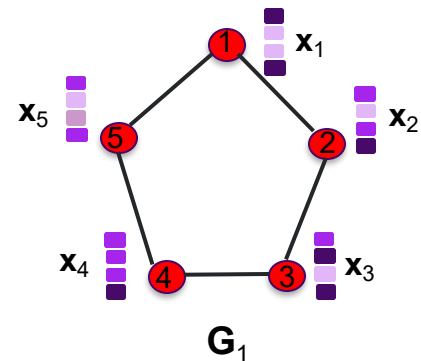


A



B

Permutation invariance and equivariance



- The order of the nodes in a graph is not fixed
- GraphNN should produce the same result for G_1 & G_2
- i.e., the result of any function/NN shouldn't depend on the node ordering

Node feature matrix:

$$X = \begin{bmatrix} \text{purple} & \text{pink} & \text{light purple} & \text{dark purple} \\ \text{pink} & \text{light purple} & \text{dark purple} & \text{purple} \\ \text{light purple} & \text{dark purple} & \text{purple} & \text{pink} \\ \text{dark purple} & \text{purple} & \text{pink} & \text{light purple} \\ \text{purple} & \text{light purple} & \text{dark purple} & \text{pink} \end{bmatrix} \begin{matrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \\ \mathbf{x}_4 \\ \mathbf{x}_5 \end{matrix} = (\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4, \mathbf{x}_5)^\top$$

Invariance: $f(PX, PAP^T) = f(X, A)$

Equivariance: $f(PX, PAP^T) = Pf(X, A)$

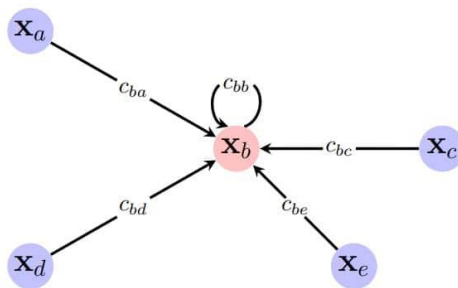
GNN variants

- $a_v^{(k)} = AGGREGATE^{(k)}\left(\left\{h_u^{(k-1)} : u \in \mathcal{N}(v)\right\}\right)$

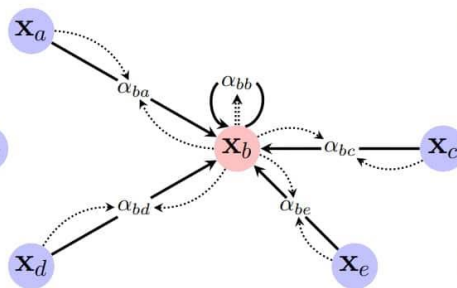
- $h_v^{(k)} = COMBINE^{(k)}\left(h_v^{(k-1)}, a_v^{(k)}\right)$

- $h_G = READOUT\left(\left\{h_v^{(K)} | v \in G\right\}\right)$

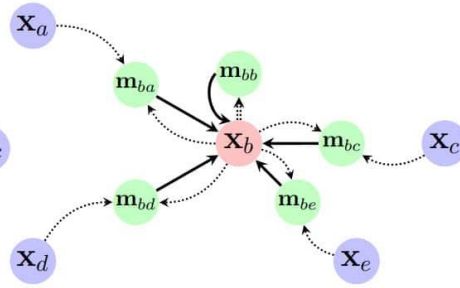
- In GCN (Kipf
 $a_v^{(k)} = MEAN$



Convolutional

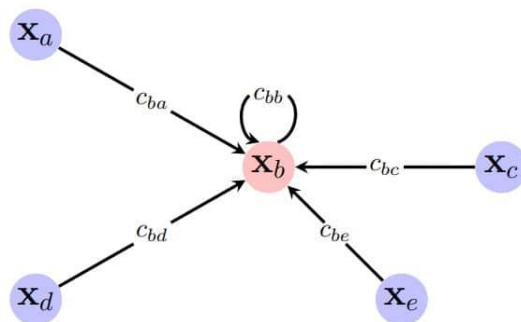


Attentional

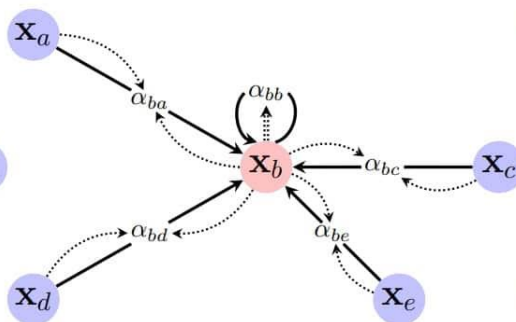


Message-passing

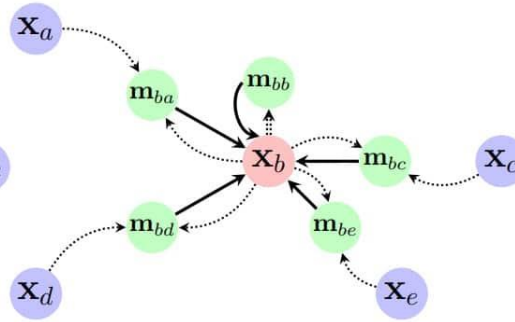
GNN variants



Convolutional



Attentional



Message-passing

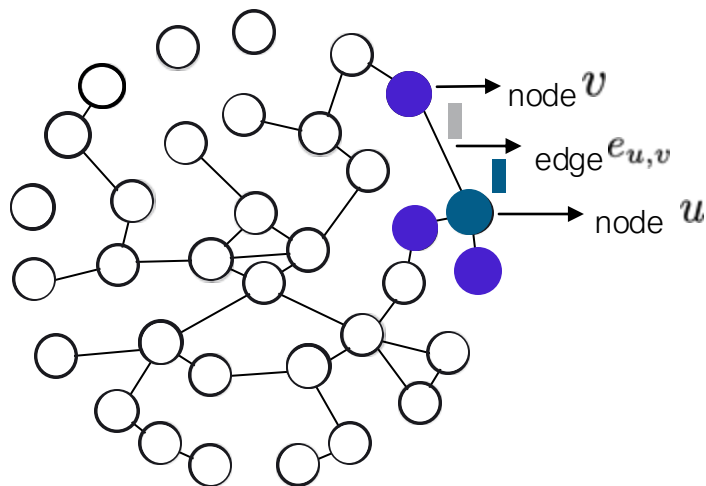
$$h_i^{(l+1)} = \phi\left(h_i^{(l)}, \bigoplus_{j \in \mathcal{N}(i)} c_{ij} \psi(h_j^{(l)})\right)$$

$$h_i^{(l+1)} = \phi\left(h_i^{(l)}, \bigoplus_{j \in \mathcal{N}(i)} \alpha(h_i^{(l)}, h_j^{(l)}) \psi(h_j^{(l)})\right)$$

$$h_i^{(l+1)} = \phi\left(h_i^{(l)}, \bigoplus_{j \in \mathcal{N}(i)} \psi(h_i^{(l)}, h_j^{(l)})\right)$$

Some graphs concepts

Graph $G = (\mathcal{V}, \mathcal{E})$ $\mathcal{V}$ set of nodes $\mathcal{E}$ set of edges



k-hop neighborhood $\mathcal{N}^k(u) = \{v | d(u, v) = k\}$

node feature $\mathbf{x}_u \in \mathbb{R}^d$

1.5	26	28	105	61.3	5.8
-----	----	----	-----	------	-----

$d = 6$

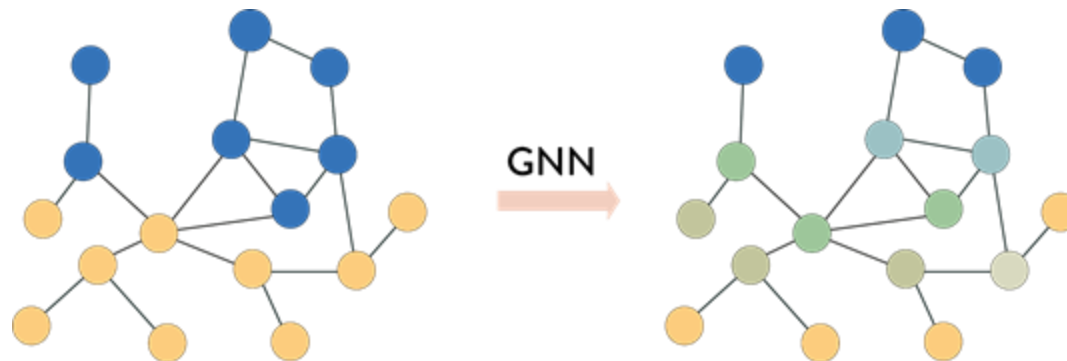
edge feature $\mathbf{x}_{u,v} \in \mathbb{R}^c$

5.3	26	105	32.1
-----	----	-----	------

$c = 4$

local neighborhood $\mathcal{N}(u)$

Prevailing GNN paradigm | Message passing

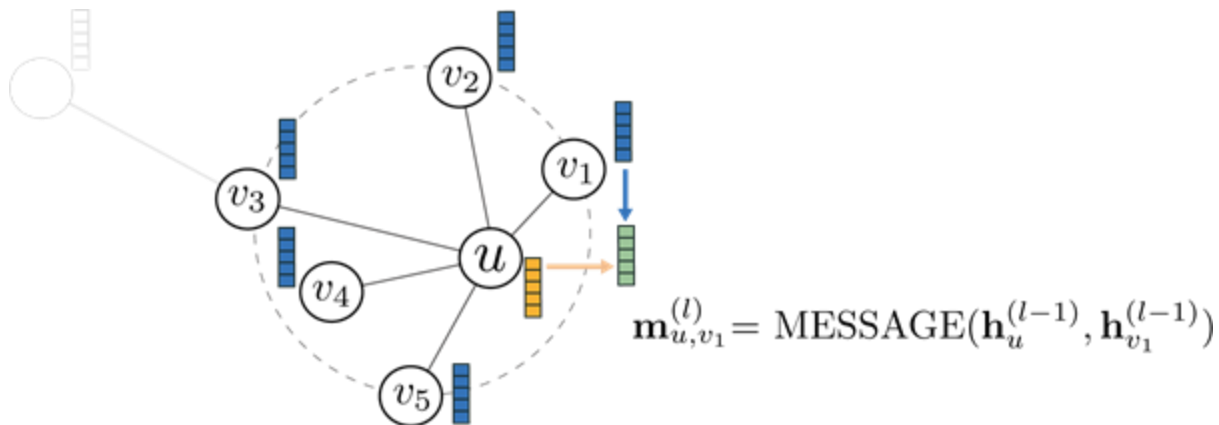


$$\mathbf{m}_{u,v_i}^{(l)} = \text{MESSAGE}(\mathbf{h}_u^{(l-1)}, \mathbf{h}_{v_i}^{(l-1)})$$

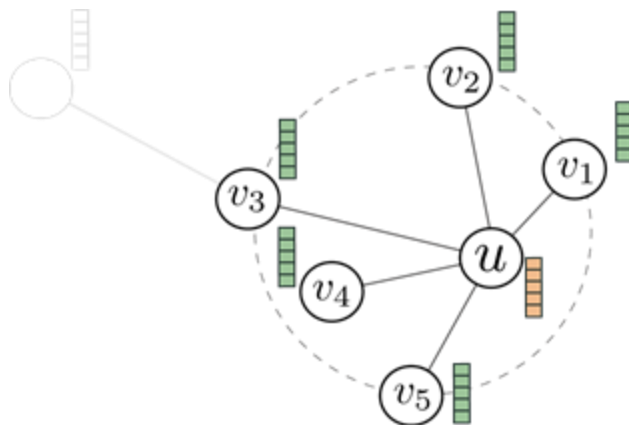
$$\mathbf{m}_u^{(l)} = \text{AGGREGATE}(\mathbf{m}_{u,v_i}^{(l)} \mid v_i \in \mathcal{N}(u))$$

$$\mathbf{h}_u^{(l)} = \text{UPDATE}(\mathbf{m}_u^{(l)}, \mathbf{h}_u^{(l-1)})$$

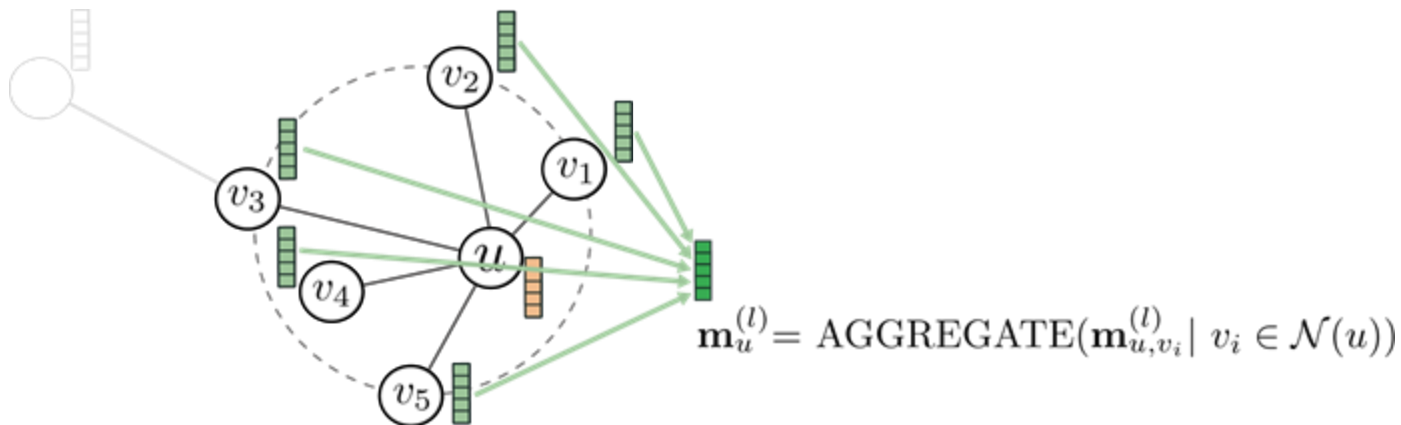
Prevailing GNN paradigm | Message passing



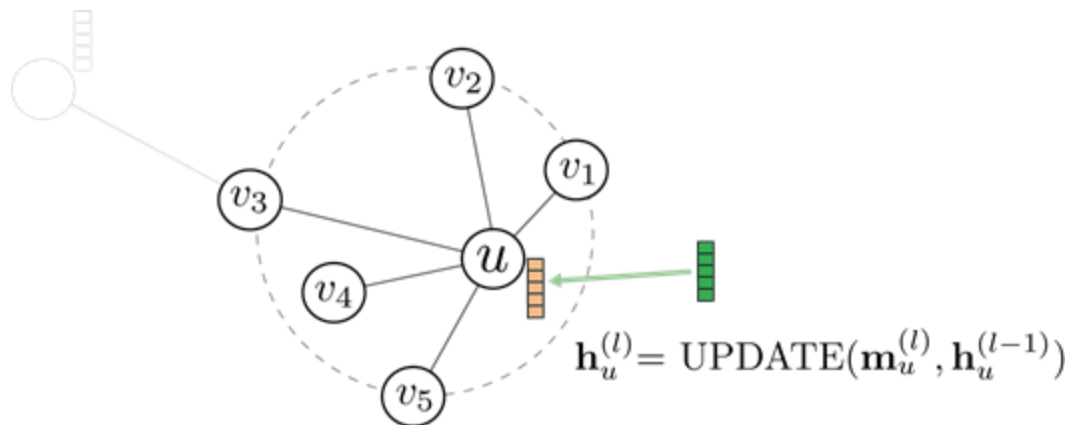
Prevailing GNN paradigm | Message passing



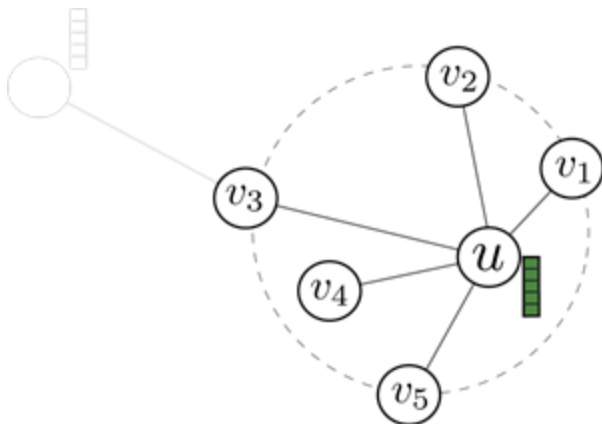
Prevailing GNN paradigm | Message passing



Prevailing GNN paradigm | Message passing



Prevailing GNN paradigm | Message passing

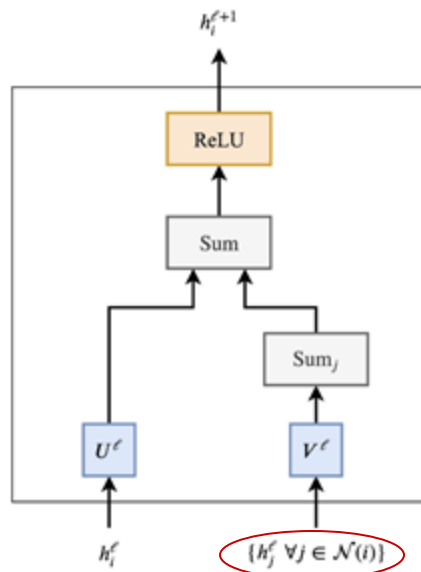
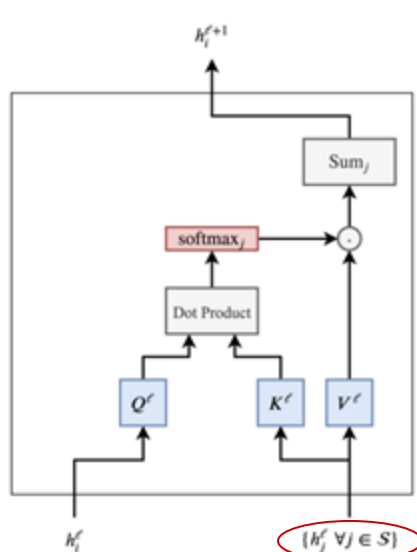


Graph transformers

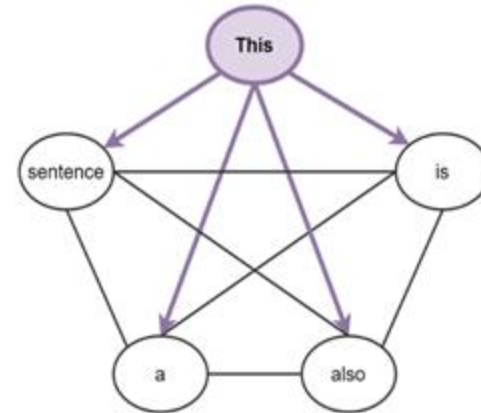
Transformers are Graph Neural Networks

Chaitanya Joshi

Feb 12, 2020



Sentences are fully-connected word graphs!



Graph transformers

Transformers are Graph Neural

|

C
F

A Generalization of Transformer Networks to Graphs

Vijay Prakash Dwivedi,[¶] Xavier Bresson[¶]

[¶] School of Computer Science and Engineering, Nanyang Technological University, Singapore
vijaypra001@e.ntu.edu.sg, xbresson@ntu.edu.sg

Graph Transformer Network: Develops a theoretical framework for generalizing the transformer model to graph data, addressing how **self-attention** and **positional encodings** can be adapted for graphs.

Graph transformers

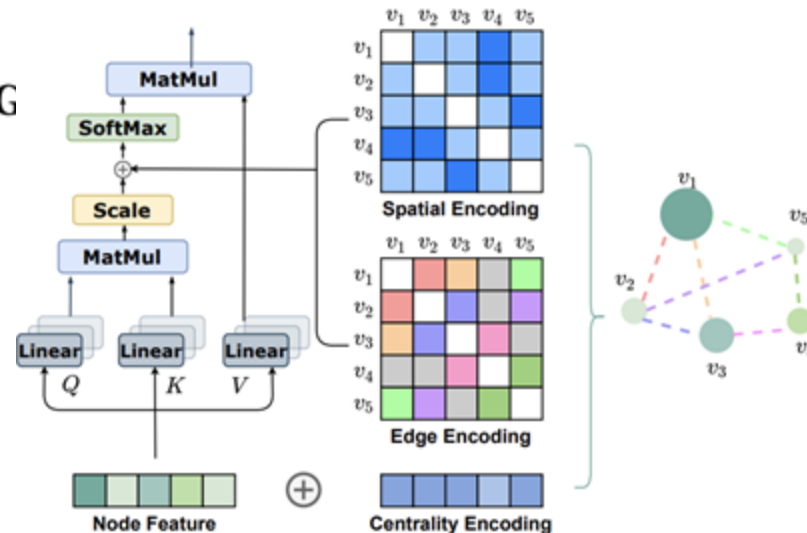
Transformers are Graph Neural

A Generalization of Transformer Networks to G

Sch

Do Transformers Really Perform Bad for Graph Representation?

Chengxuan Ying¹; Tianle Cai², Shengjie Luo³;
Shuxin Zheng⁴, Guolin Ke⁴, Di He⁴, Yanming Shen¹, Tie-Yan Liu⁴
¹Dalian University of Technology ²Princeton University
³Peking University ⁴Microsoft Research Asia
yingchengxuan@gmail.com, tianle.cai@princeton.edu, luosj@stu.pku.edu.cn
{shuz, guoke, dihe, tyliu}@microsoft.com, shen@dlut.edu.cn



Graphormer introduces **spatial encoding** to capture graph structure, **edge encoding** to incorporate edge features into the attention mechanism, and **centrality encoding** to enhance node importance representation.

Limitations of GNNs

- To capture long range information, we need GNNs with sufficient depth
- Current GNNs suffer from performance degradation at higher depth, which limits their application
- This degradation happens because:
 - Over-smoothing (nodes' features become similar to each other as # layers $\rightarrow \infty$)
 - Over-squashing (graph bottlenecks cause messages between distant nodes to become overly-squashed)

Resources for implementing GNNs



2.7.0

[PyG Documentation](#)

PyG Documentation

 **PyG (PyTorch Geometric)** is a library built upon  **PyTorch** to easily write and train Graph Neural Networks (GNNs) for a wide range of applications related to structured data.



[About](#) [Get Started](#) [Tutorials](#)

DEEP GRAPH LIBRARY

Easy Deep Learning on Graphs

Install

GitHub

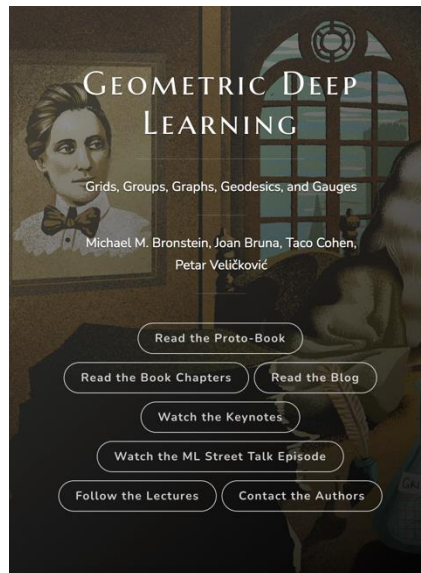
Name	SparseTensor	edge_weight	edge_attr	bipartite	static	lazy
SimpleConv	✓	✓		✓	✓	
GCNConv (Paper)	✓	✓			✓	✓
ChebConv (Paper)		✓			✓	✓
SAGEConv (Paper)	✓			✓	✓	✓
CuGraphSAGEConv (Paper)					✓	
GraphConv (Paper)	✓	✓		✓	✓	✓
GatedGraphConv (Paper)	✓	✓			✓	
ResGatedGraphConv (Paper)	✓		✓	✓	✓	✓
GATConv (Paper)	✓		✓	✓		✓
CuGraphGATConv (Paper)					✓	
FusedGATConv (Paper)					✓	
GATv2Conv (Paper)	✓		✓	✓		✓

Reading list

- Li, M.M., Huang, K. & Zitnik, M. Graph representation learning in biomedicine and healthcare. Nat. Biomed. Eng 6, 1353–1369 (2022). URL: <https://www.nature.com/articles/s41551-022-00942-x>
- Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A Comprehensive Survey on Graph Neural Networks. 1 2019. URL: <http://arxiv.org/abs/1901.00596>
- Thomas N. Kipf and Max Welling. Semi-Supervised Classification with Graph Convolutional Networks. 9 2016. URL: <http://arxiv.org/abs/1609.02907>
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph Attention Networks. 10 2017. URL: <http://arxiv.org/abs/1710.10903>
- William L Hamilton, Rex Ying, and Jure Leskovec. Inductive Representation Learning on Large Graphs. URL: <https://arxiv.org/abs/1706.02216>
- Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling Relational Data with Graph Convolutional Networks. 3 2017. URL: <http://arxiv.org/abs/1703.06103>.

Related reading

- Review on current geometric ML approaches for structure-based small molecule design: <https://arxiv.org/pdf/2306.11768>
- RNNs explained: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>
- Great explanation (intuitive and with equations) of transformers and their connection with GNNs: <https://graphdeeplearning.github.io/post/transformers-are-gnns/>
- Introduction to graph representation learning and GNNs: <https://prescient.gene.com/blog/2023-08-16-graph-representation-learning/>



Prescient
Design

A Genentech Accelerator



Generative AI

In Drug Design

Genentech

A Member of the Roche Group

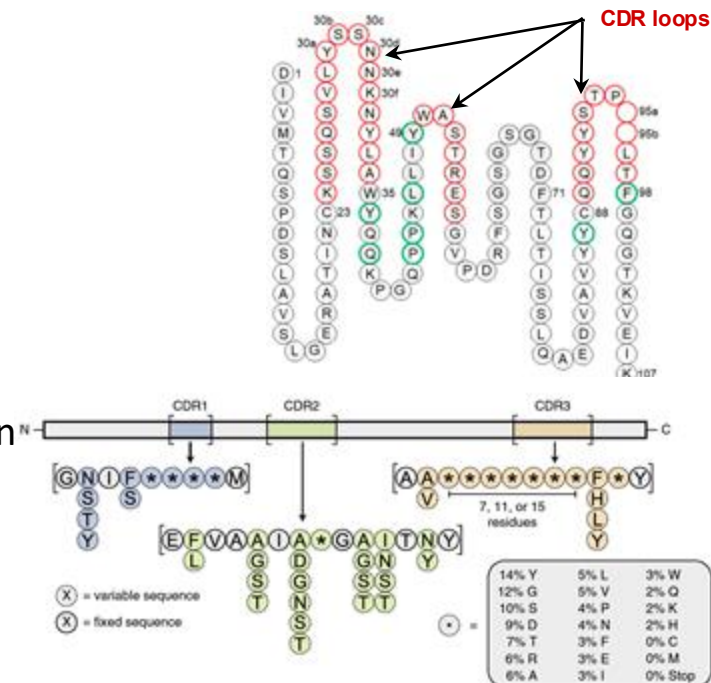
Computational protein design/optimization problem

Key challenge: Searching through a *prohibitively large* combinatorial discrete space, $\mathcal{S}$, for a solution that max scoring function, $f(s)$:

$$\operatorname{argmax}_{s \in \mathcal{S}} f(s)$$

Problem: $\mathcal{S}$ is sparsely functional

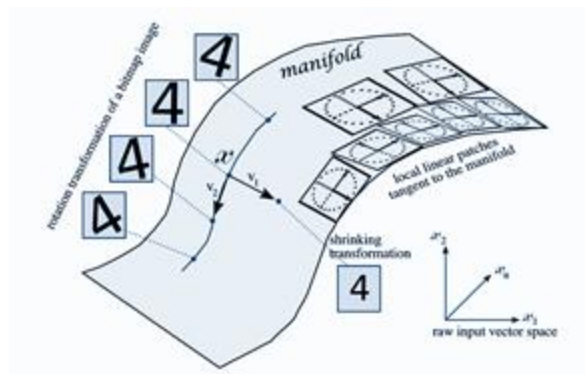
- Rational design (David Baker) and directed evolution (Frances Arnold)



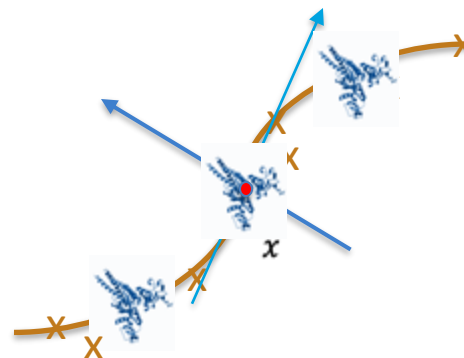
McMahon, C., ... & Kruse, A. C. (2018), *Nature structural & molecular biology*.

The Curse of Dimensionality and the Manifold Assumption

Real-world, high-dimensional data lie (roughly) on a low-dimensional manifold (Chapelle 2006)



Bengio (2009)



Instead of the **(large)** combinatorial space, search for a novel sequence $\mathbf{x}$ is restricted on the **(small)** manifold space, i.e., $\mathbf{x} \in \mathcal{M}$

Function-guided design: $\operatorname{argmax}_{\mathbf{x} \in \mathcal{M}} f(\mathbf{x})$ for a scoring function $f(\mathbf{x})$ trained on $\mathcal{M}$ where $f(\mathbf{x})$ is an oracle.

Learning and Sampling from the Data Manifold: Denoising Autoencoders (DAE)

Vincent et al. (2010) introduced DAEs:

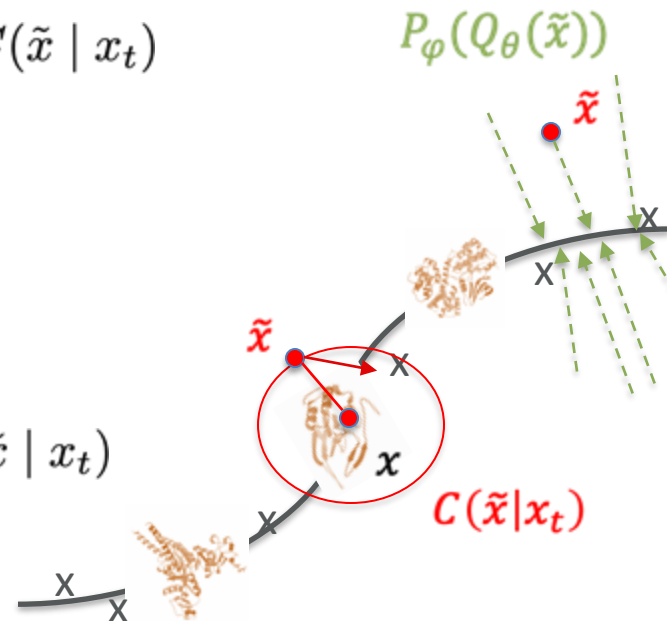
$$\min_{\varphi, \theta} \frac{1}{N} \sum_{t=1}^N \|x - P_{\varphi}(Q_{\theta}(\tilde{x}_t))\|^2 \text{ where } \tilde{x}_t \sim C(\tilde{x} | x_t)$$

Encoder
Decoder

and $C(\tilde{x} | x_t)$ is unstructured noise centered on x_t .

Then $x' \sim P(x) \iff x' \sim P_{\varphi}(Q_{\theta}(\tilde{x}_t))$, where $\tilde{x}_t \sim C(\tilde{x} | x_t)$

Admits **Markov Chain Monte Carlo (MCMC)**-like iterative sampling procedure



NeurIPS 2021 Machine Learning for Structural Biology -
Oral Presentation

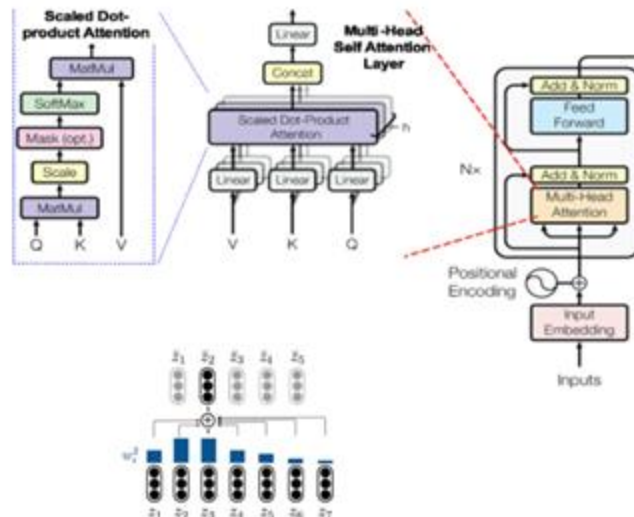


Figure 2: Illustration of the length transformation mechanism.

How Generative AI can help in Sequence/Structure Design

Generative models in a nutshell.

Learn $p(\mathbf{x}) = p(x_1, x_2, \dots, x_L)$

$$p(\mathbf{x}) \approx p_{\text{data}}(\mathbf{x})$$

Alignment-based models:

- Hidden Markov Models (HMM)
- Potts model

Latent variable models:

- GANs (e.g., Namrata *et al.*, 2018)
- VAEs (e.g., Riesselman *et al.*, 2018)

Language models:

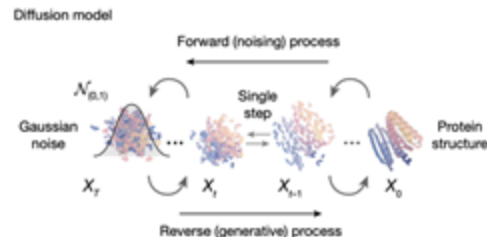
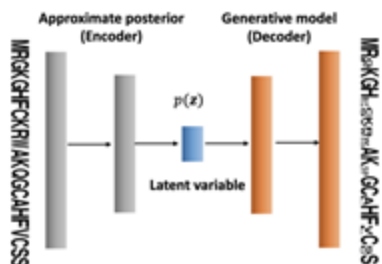
- ProGen (e.g., Madani *et al.*, 2020)
- Manifold Sampler (Gligorijevic, et al. 2021)

Diffusion Models

- RFDiffusion (Chroma, 2023)
- WJS, AbDiffuser, (Prescient, 2023)

such that:

```
AREALVPFG-- -KDAPLLIIV TVLTSMEASD LVD-LGMTLS  
SREILEPYG-- -KERPLLIGV TVLTSMESAD LQG-IGILSA  
VVNALQSITL- -KEKPLLIGV TILTSLDGSD LKT-LGIQEK  
AKDGLIAGTPA GHSVPKLLAV TELTSISDDV LRNEQNCRLP
```



Processes sequence in one direction



$$p(x_i = A | x_1 \dots x_{i-1})$$

Model: Overview

Given a protein (target) sequence $Y = y_1, \dots, y_n$, learn a Seq2Seq DAE model $X \rightarrow Y$ where X is a corrupted version of Y : $\mathbf{X} \sim C(\mathbf{X}|\mathbf{Y})$

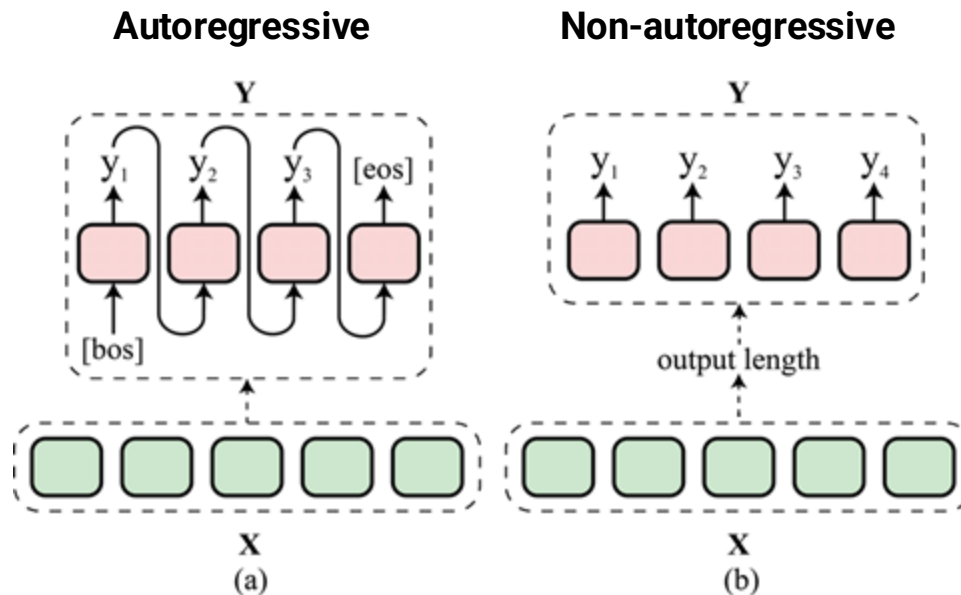
Two approaches:

1. Autoregressive Seq2Seq:

$$\log p(\mathbf{Y}|\mathbf{X}) = \sum_t \log p(y_t, y_{<t}, \mathbf{X})$$

1. Non-autoregressive Seq2Seq:

$$\log p(\mathbf{Y}|\mathbf{X}) = \sum_t \log p(y_t, \mathbf{X})$$



Non-autoregressive sampling. Makes changes in multiple positions of a target sequence enabling **effective** exploration of the overall fitness landscape and resulting in **diverse** sequence designs

Approaches to Conditional Design (exploring original sequence space)

Guiding design by an oracle (e.g., Yang *et al.* 2019, Alley *et al.* 2019, Brookes *et al.* 2019, Biswas *et al.* 2021)



Machine-learning-guided directed evolution for protein engineering

Kevin K. Yang, Zachary Wu and Frances H. Arnold*



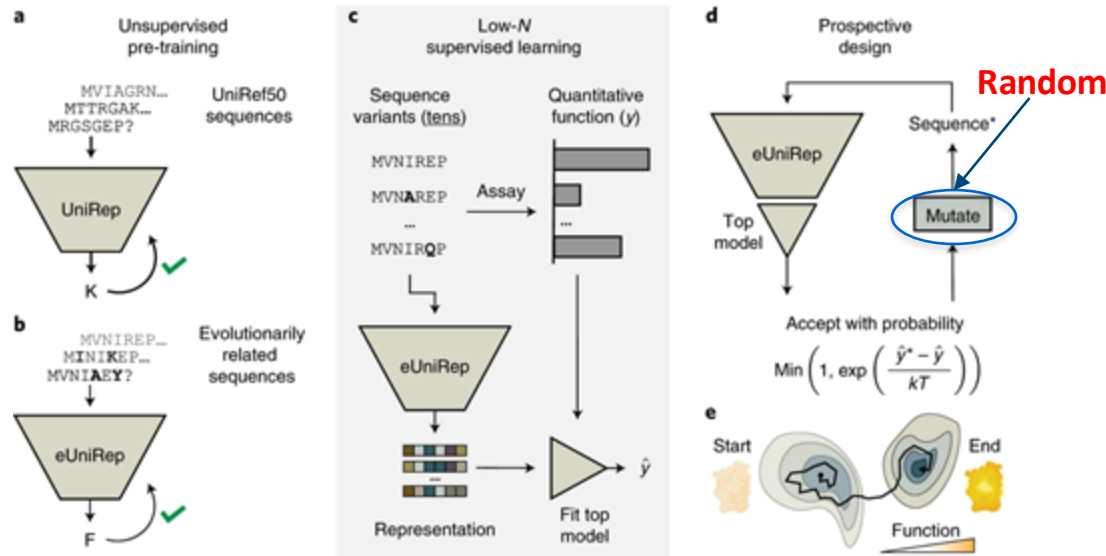
Unified rational protein engineering with sequence-based deep representation learning

Ethan C. Alley^{1,2,4}, Grigory Khimulya^{1,2}, Surojit Biswas^{1,3,4}, Mohammed AlQuraishi^{1,4} and George M. Church^{1,3,4}



Low-*N* protein engineering with data-efficient deep learning

Surojit Biswas^{1,2,4}, Grigory Khimulya^{1,2}, Ethan C. Alley^{1,4}, Kevin M. Esvelt^{1,3,4} and George M. Church^{1,3,4}

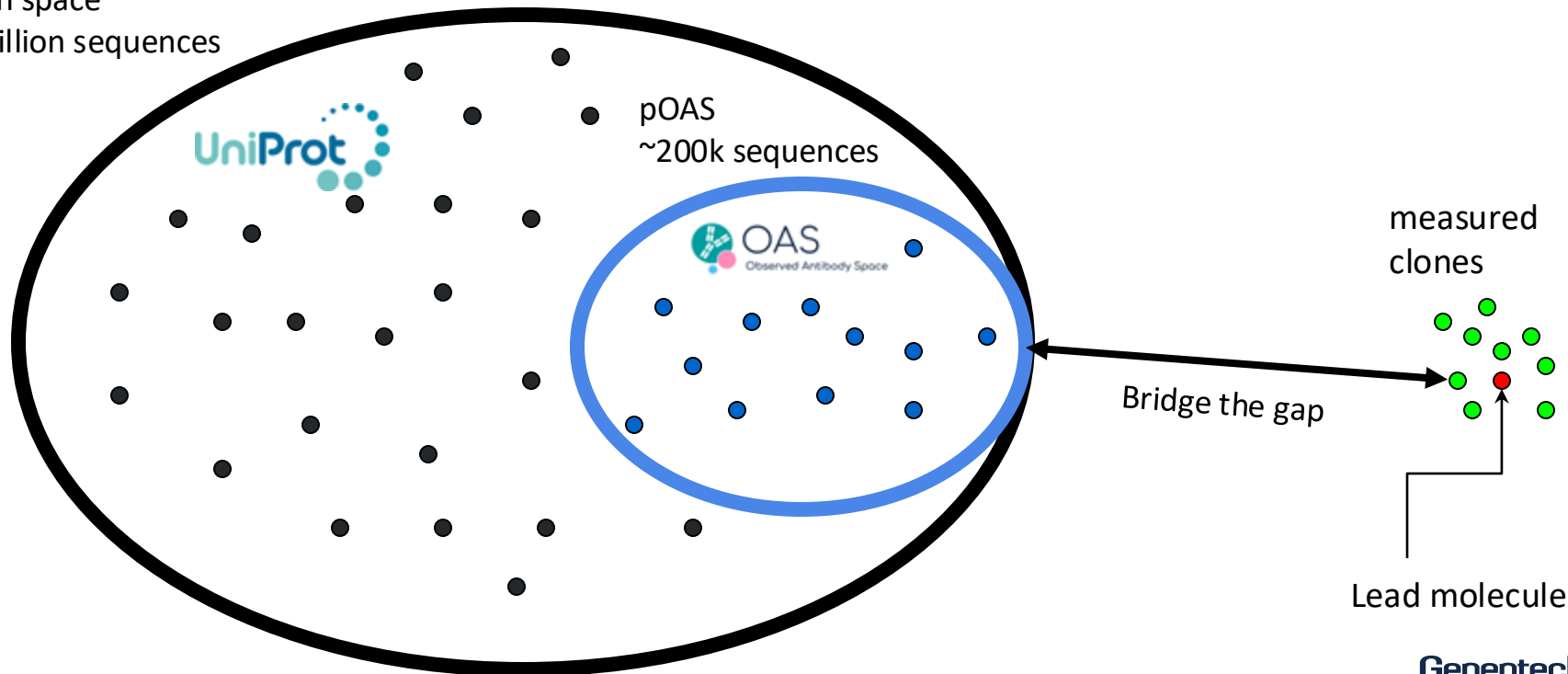


Biswas *et al.*, 2021

Publicly available data vs. reality

Protein space

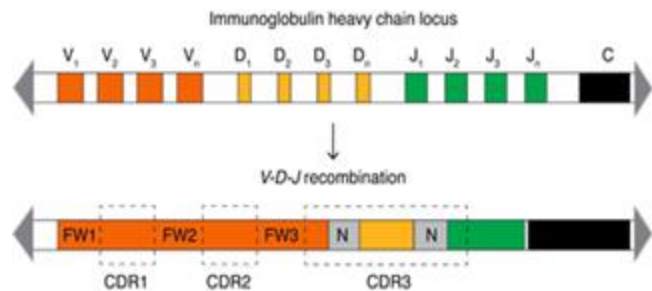
~2.4 billion sequences



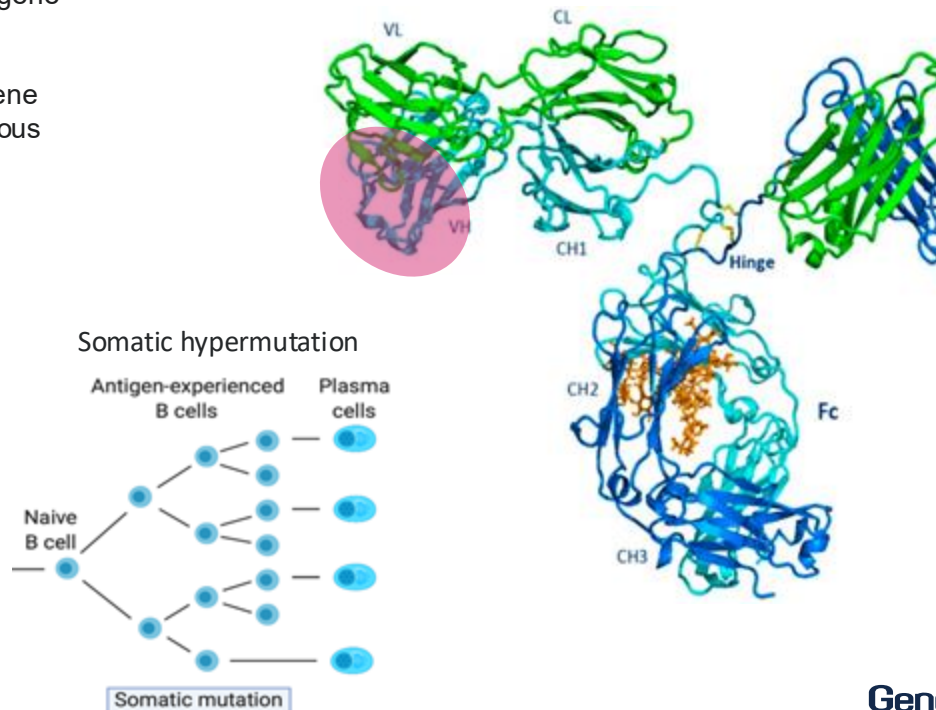
How nature generates antibodies

Genome contains multiple copies of 3 different types of gene segments for the variable regions of Abs

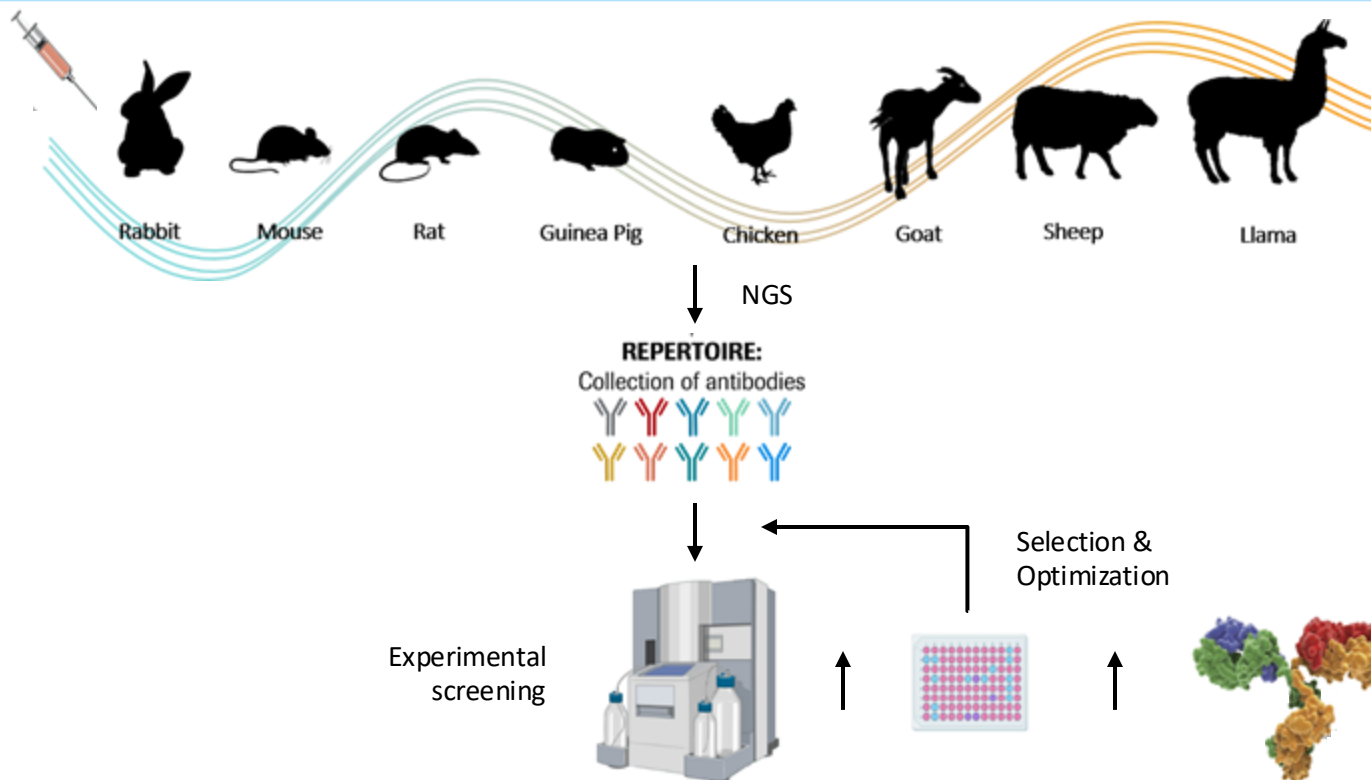
DNA rearrangement causes one copy of each type of gene segment to go in any given B-cell, generating an enormous antibody repertoire; roughly 3×10^{11} .



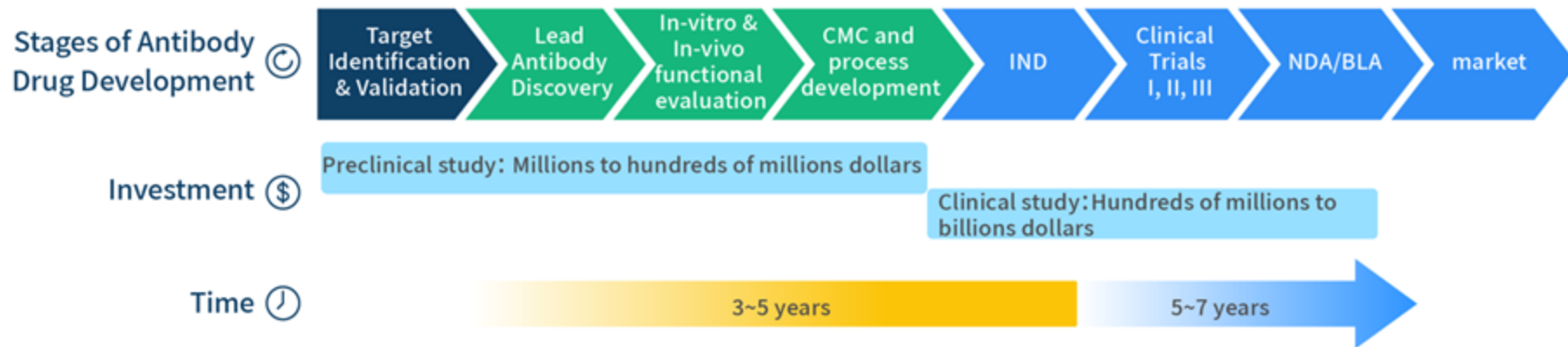
1987 Nobel Prize in Physiology or Medicine
by Susumu Tonegawa



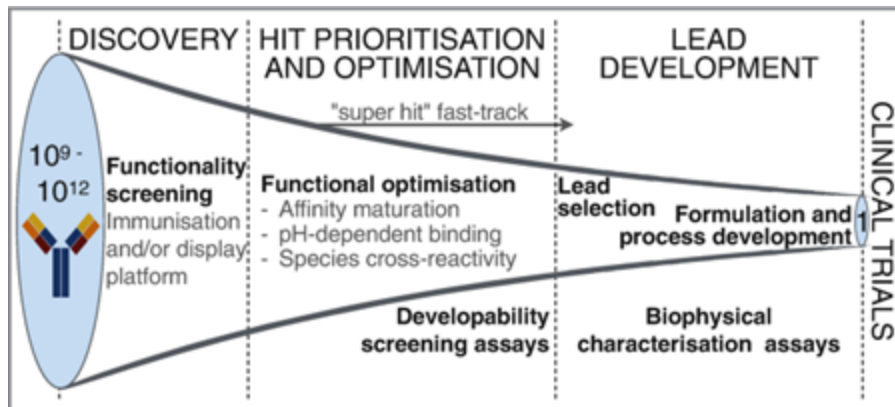
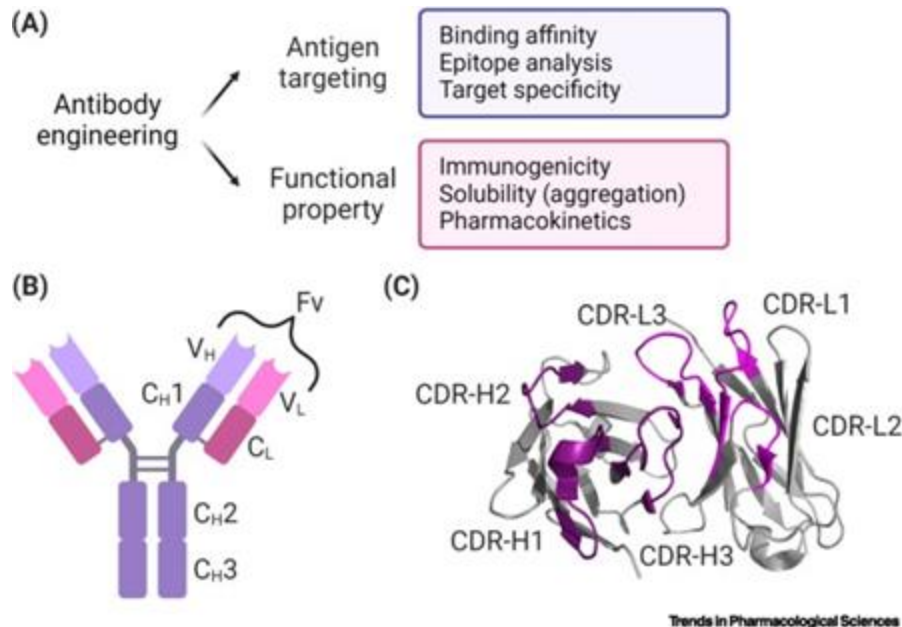
How pharma typically discovers antibodies



Stages of antibody drug development process



Antibody developability properties



How AI can help antibody design & discovery ?

Generative models:

- can help in hit finding and generating proposals (clever library design instead of combinatorial search or random diversification)
- sequence multi-property optimization (generating proposal that can satisfy multiple objectives at the same time)
- *de novo* design (that might replace the entire immunization/discovery process)

Discriminative models (property predictors/oracles):

- antibody-antigen affinity prediction (Kd predictors)
- developability property predictors for *in silico* screening (faster & cheaper)
- replacing functional assays
- guiding generative models (e.g., guiding library design)

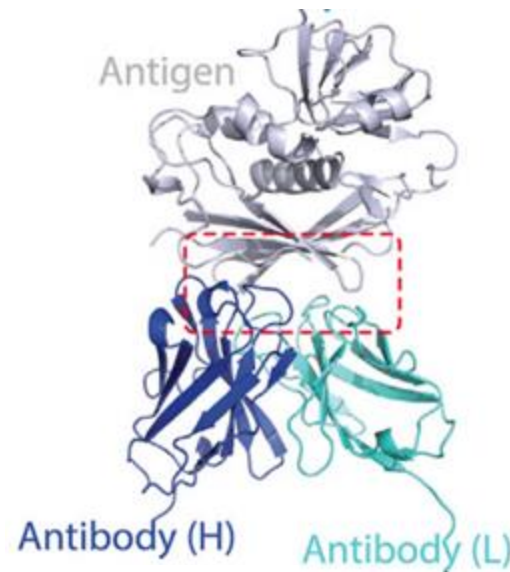
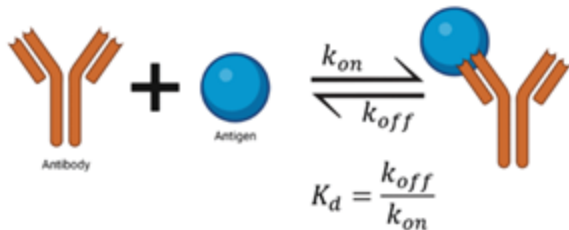
Affinity/binding prediction

Predict binding affinity

- Input type: unbound Ab+Ag (sequence or folded structures)
- Output type: (regression) equilibrium dissociation constant K_d ; (classification) bind/no bind?

Related tasks:

- Predict interface (epitope/paratope)
- Docking



Experimental techniques:

- enzyme-linked immunosorbent assay (ELISA)
- surface plasmon resonance (SPR)

Why is the problem hard ?

Machine learning



Sequence space is extremely sparsely functional

Formally: hard to learn an affinity prediction function that is Lipschitz continuous

$f : D \rightarrow \mathbb{R}$ is Lipschitz continuous

$$\iff \exists L \geq 0 \forall x, \tilde{x} \in D : |f(x) - f(\tilde{x})| \leq L \cdot |x - \tilde{x}|$$

Biology

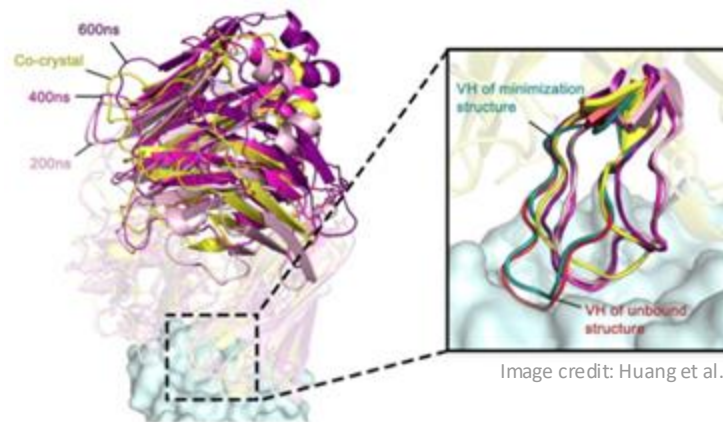
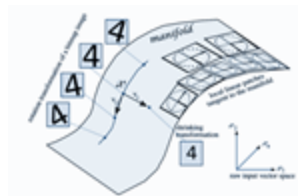


Image credit: Huang et al. 2022

Function depends on dynamics of loops and solvent, which are hard to simulate

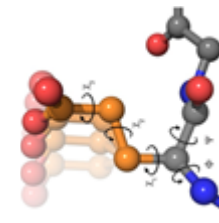
How we overcome these challenges with ML ?

Real-world high-dimensional data lie (roughly) on a **low-dimensional manifold** (Chapelle 2006). Easy to learn with deep learning.

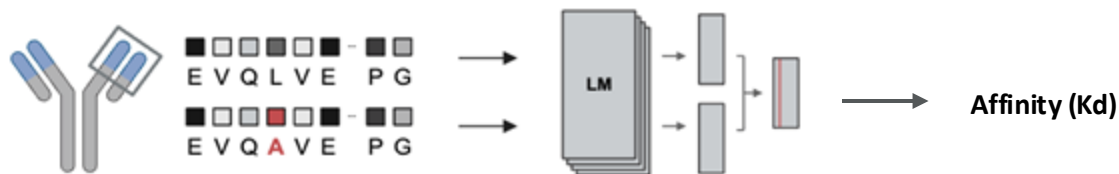


Protein representation & inductive biases help deal with data scarcity:

- pLM embeddings, equivariant models, internal geometry

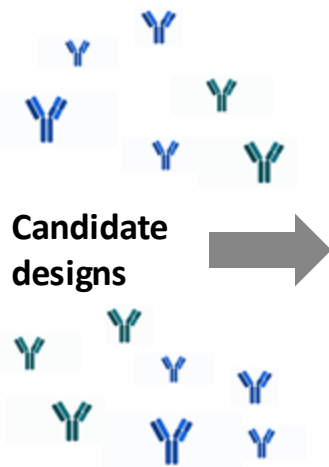


Sequence-based antibody-antigen affinity prediction



Multi-objective optimization for lead molecule optimization

How do we design molecules with better properties to: 1) reduce time-to-clinic; 2) increase clinical success rates; and 3) target previously undruggable diseases



Many (competing) properties to optimize for

How much of it can we produce?

Expression yield

Does it stick to things other than the target antigen?

Specificity

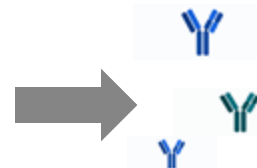
Stability

Binding affinity

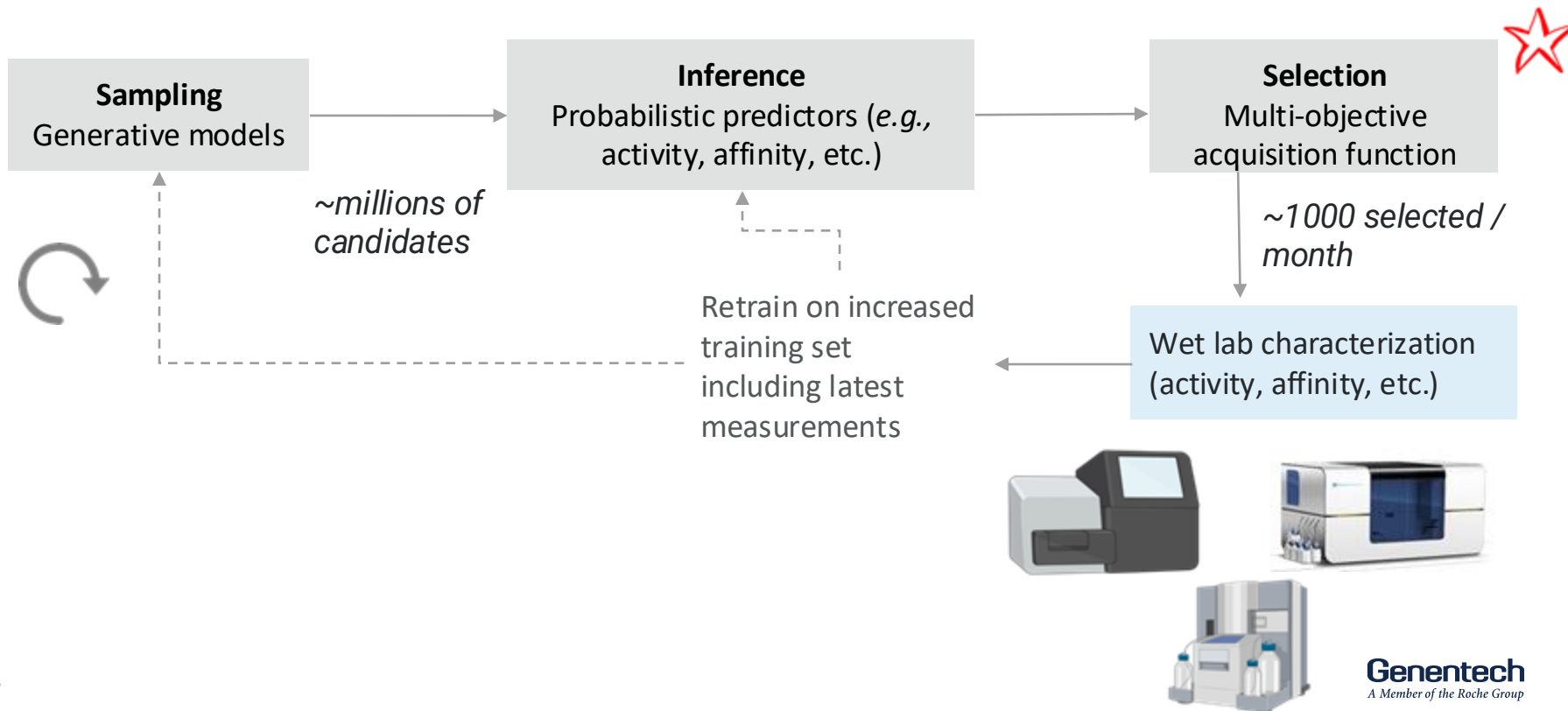
Immunogenicity

How well does it bind to the target antigen?

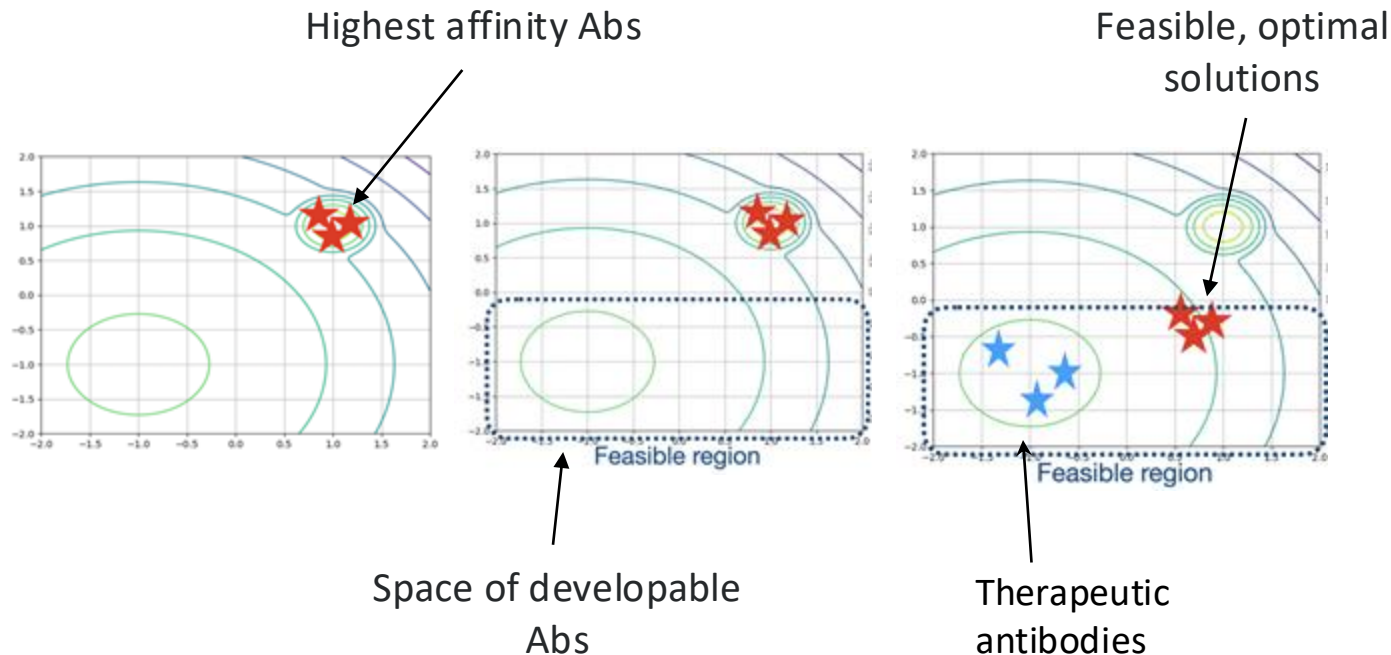
Self-association



Lab-in-the-Loop: active learning selection and , optimizing for multiple properties



Space of developable antibodies



The only way of making **therapeutic antibodies** is considering all the objective functions at the same time!

Pareto optimality & therapeutic antibodies

Find set of sequences at Pareto front:

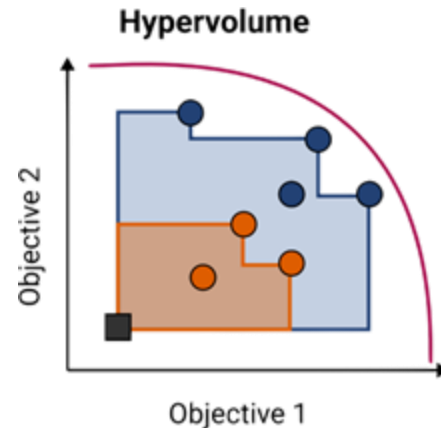
$$\mathcal{P} = \{S \mid \{S' \mid S' \succ S\} = \emptyset\}$$

$S' \succ S$ if $f_k(S') > f_k(S)$ for all $k = 1, \dots, K$
where, $f_k(S) \in \mathbb{R}$ is the k -th objective function

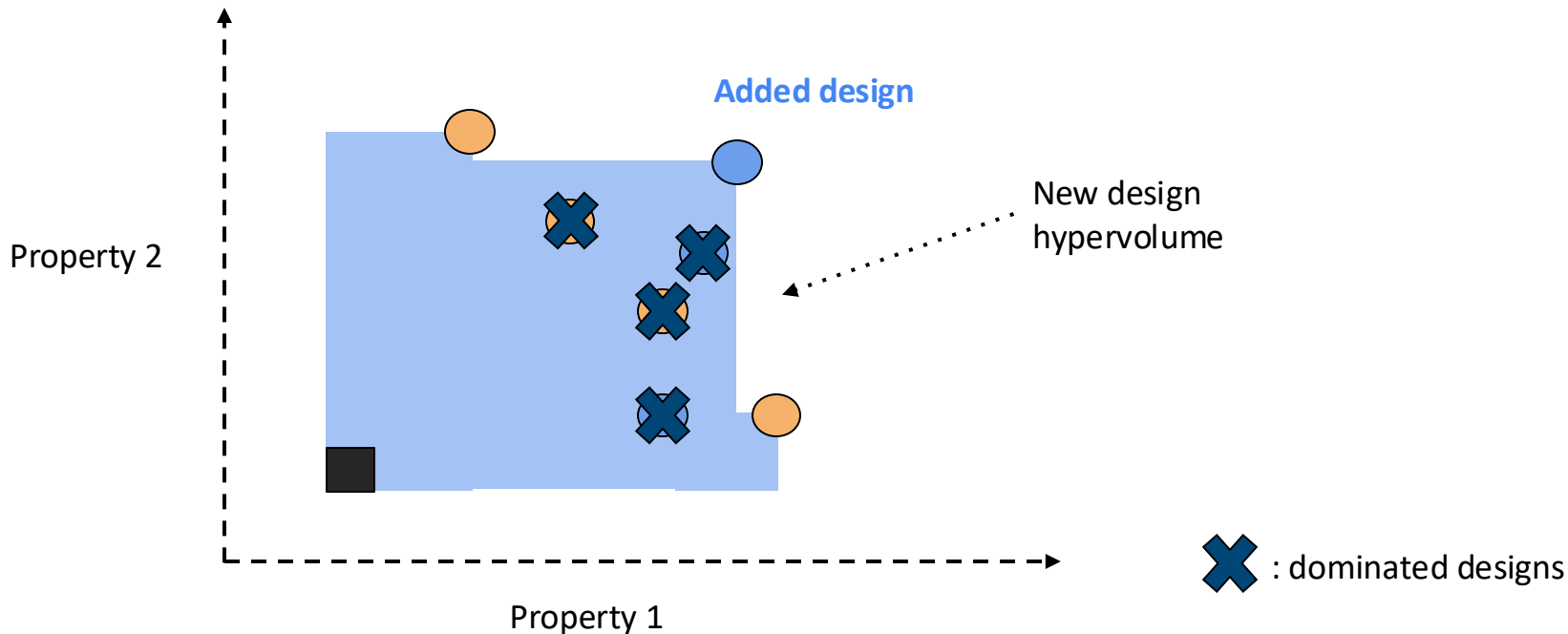
(S' strictly dominates S)
(affinity, activity, specificity, etc.)

How to measure the quality of the frontier (acquisition function)?

- Volume: Expected hypervolume improvement (EHVI; Emmerich *et al.* 2011)
- Expected HVI is defined over the posterior distribution of our property predictors (pseudo-oracles) (Daulton *et al.*, 2020)



Multi-property optimization: The best designs increase the “hypervolume” the most

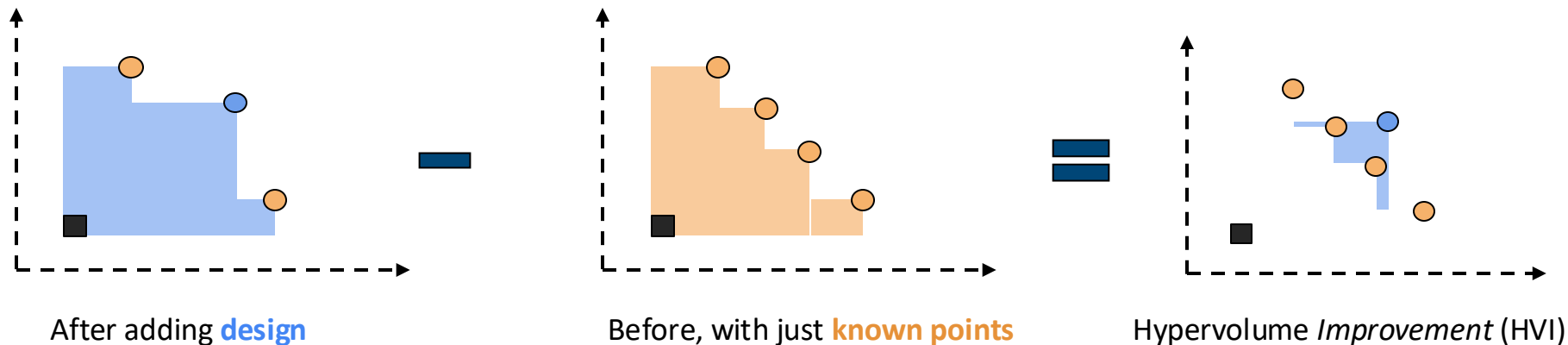


Properties are **jointly** maximized!

Contrast with sequential maximization or arbitrary weighting of properties

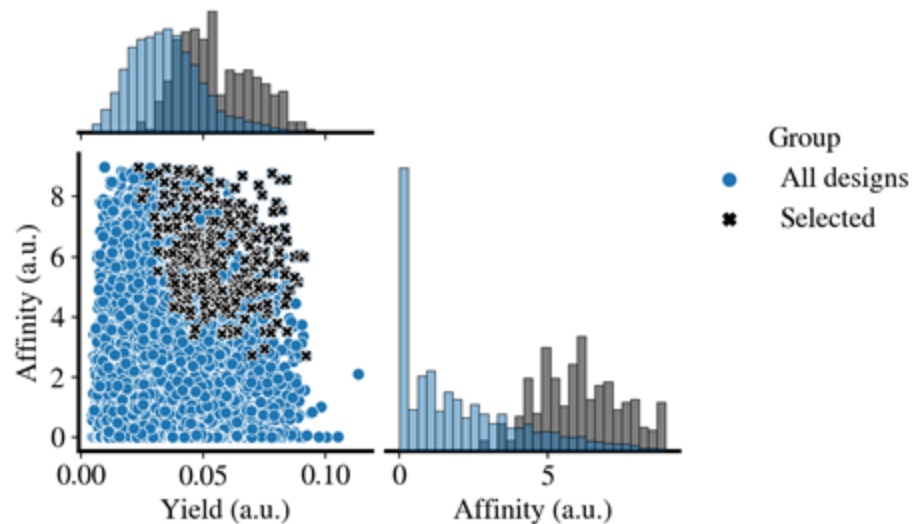
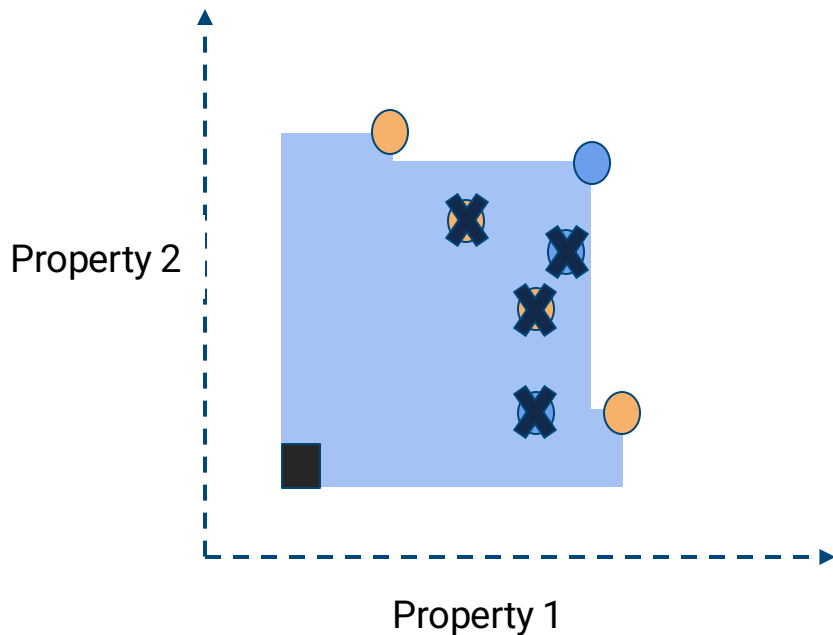
What is hypervolume *improvement*?

Take the difference of the hypervolumes before and after adding the candidate design.
The best designs have the highest HVI.

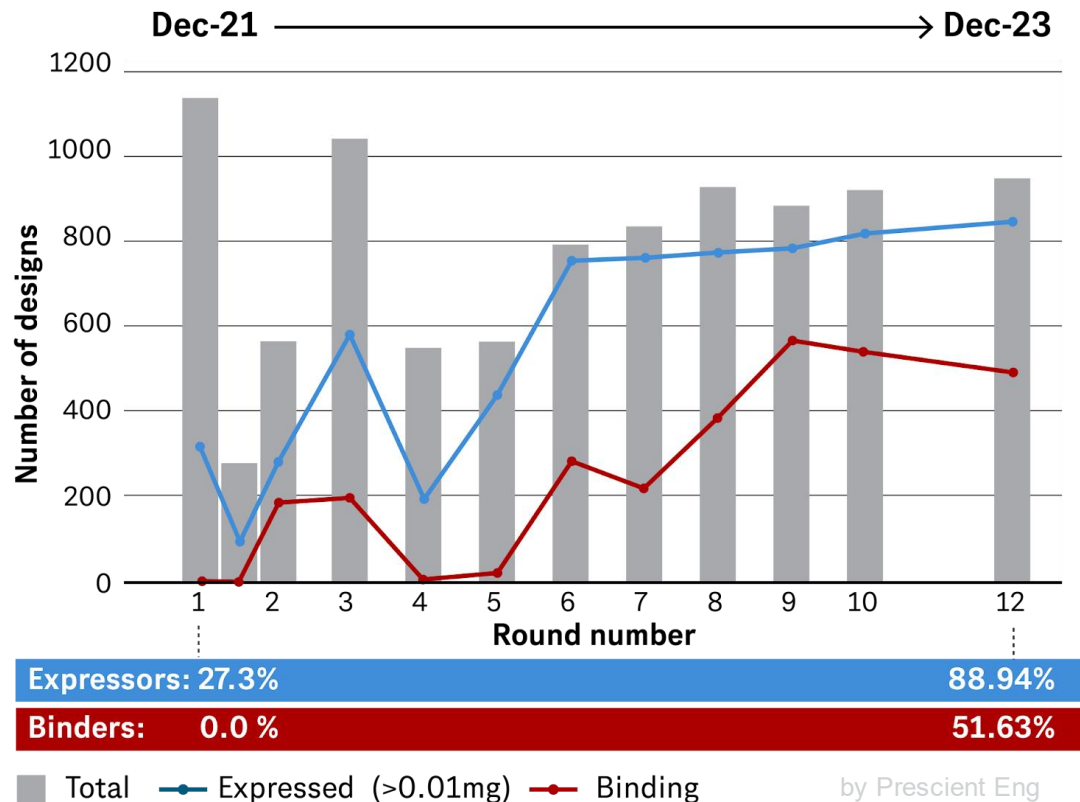


Active learning: multi-objective sample selection through expanding the Pareto frontier

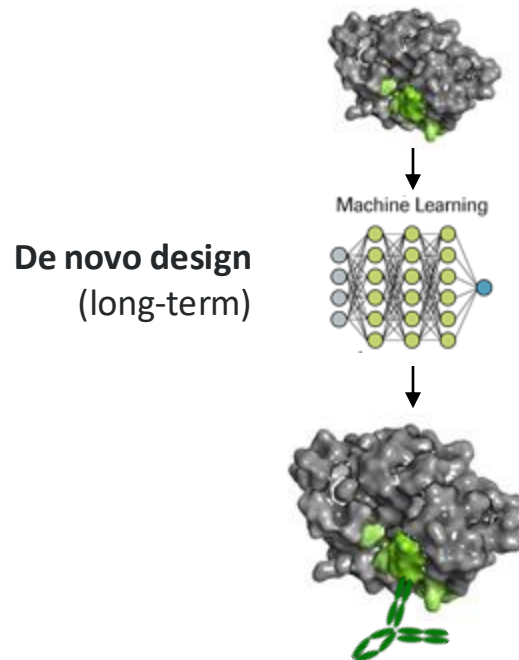
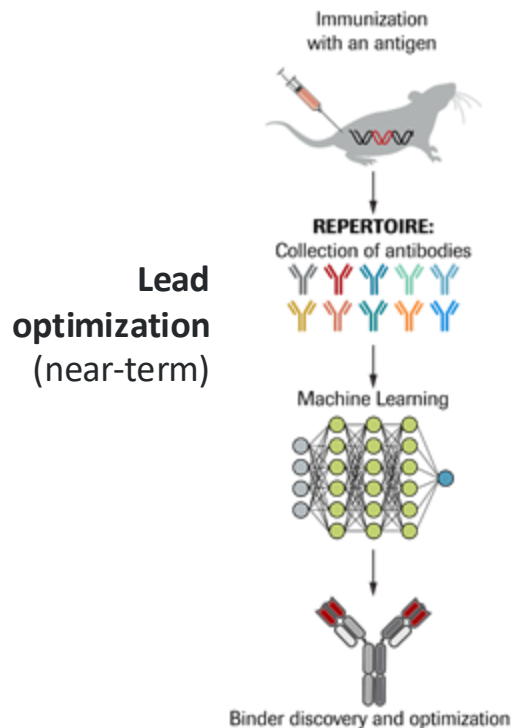
BOtied: Expected hypervolume improvement (EHVI) guides exploration vs. exploitation balance towards expanding the Pareto frontier over multiple developability properties.



LitL importance evidenced by expression & affinity round-over-round improvement

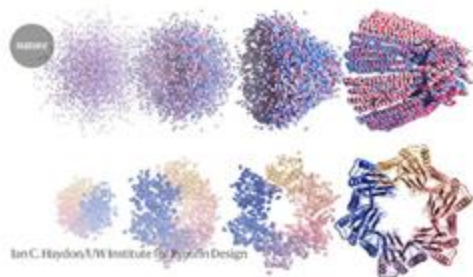


How will *de novo* generative AI impact antibody discovery in the future ?

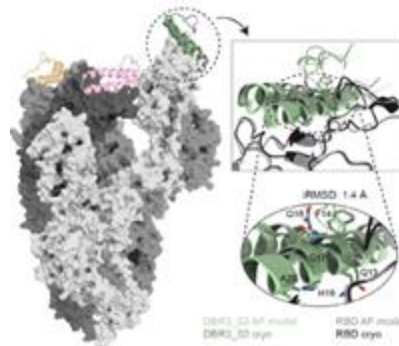


Great progress in protein design with generative AI

General protein design

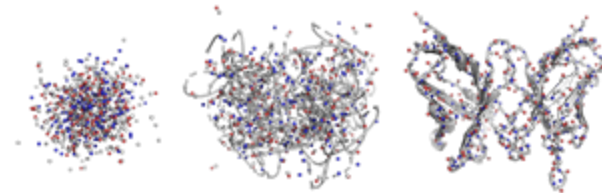


De novo design of protein structure and function with **RFdiffusion**. Watson et al. 2023



De novo design of site-specific protein interactions with learned surface fingerprints. Gainza et al. Nature 2023 (**MaSIF**)

Antibody-specific models

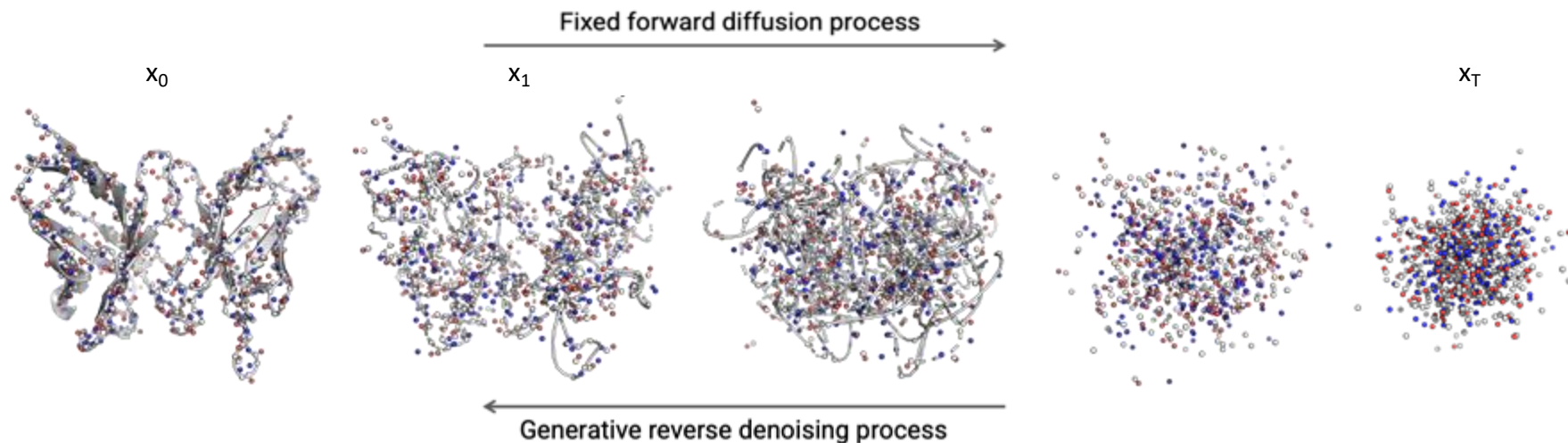


- Iterative Refinement Graph Neural Network for Antibody Sequence-Structure Co-design. ICLR 2022 (**RefineGNN**)
- *Protein Structure and Sequence Generation with Equivariant Denoising Diffusion Probabilistic Models*. Namrata & Tudor. arXiv 2022
- End-to-End Full-Atom Antibody Design. ICML 2023 (**dyMEAN**)
- Antigen-Specific Antibody Design and Optimization with Diffusion-Based Generative Models. Luo et al. NeurIPS 2023. (**DiffAb**)

Ingraham et al 2022 (**Chroma**), Yim et al 2023 (**FrameDiff**), ..

Diffusion dominates!

Denoising Diffusion on proteins

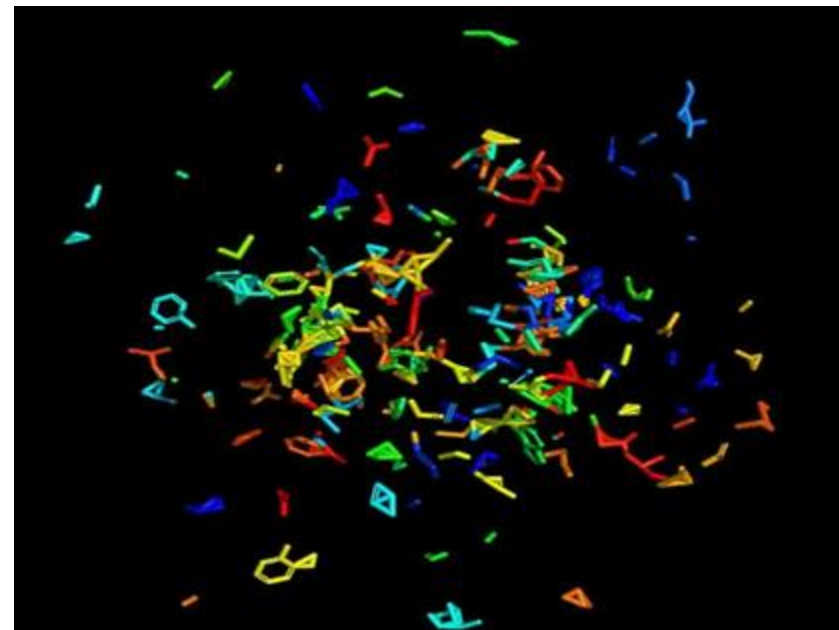
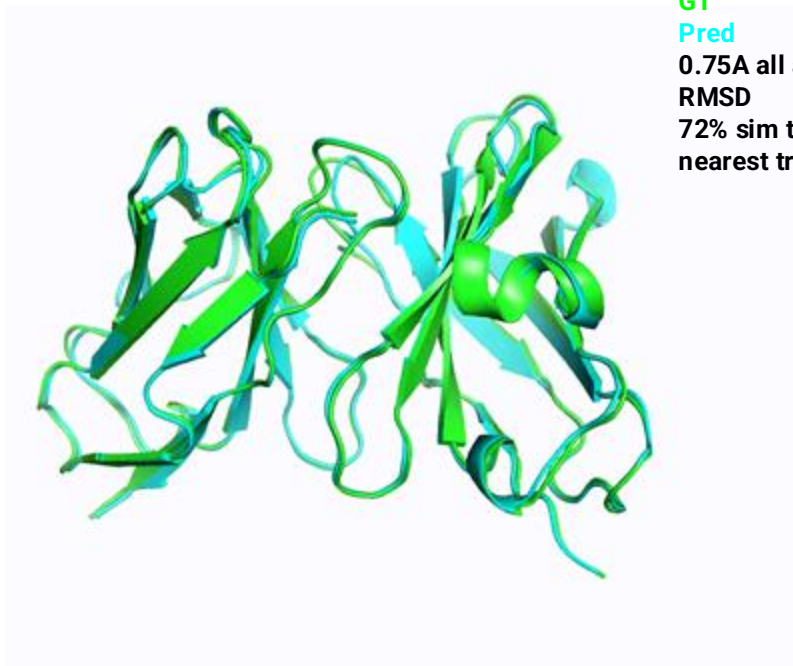


Steps from a sampling trajectory. Beginning with randomly initialised positions, our model iteratively updates the position of the backbone atoms, producing a recognisable antibody backbone at the end of the process.

Martinkus, Karolis, et al. "Abdiffuser: Full-atom generation of in-vitro functioning antibodies." *NeurIPS* (2023).

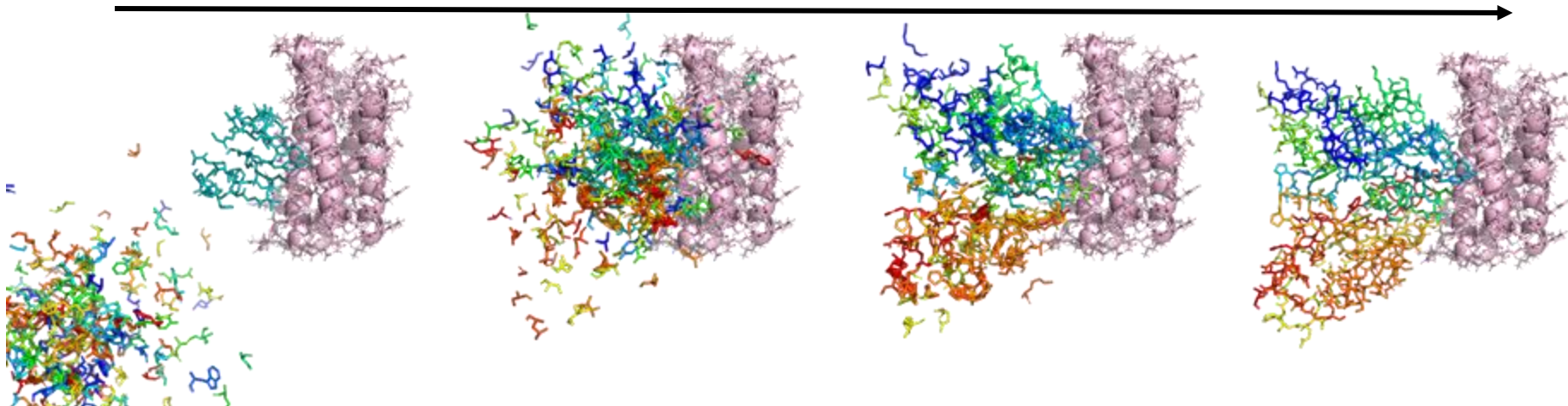
EquiFold: antibody structure prediction & design

7oo2_CD
GT
Pred
0.75A all atom
RMSD
72% sim to
nearest training



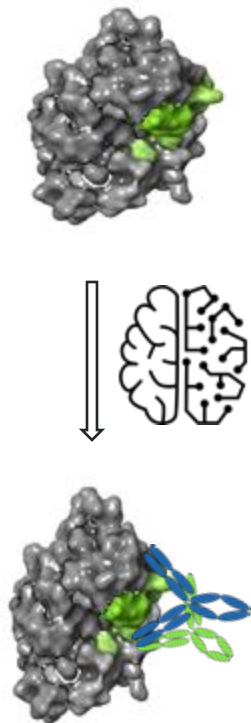
Guided diffusion sampling

Direction of guided generative process

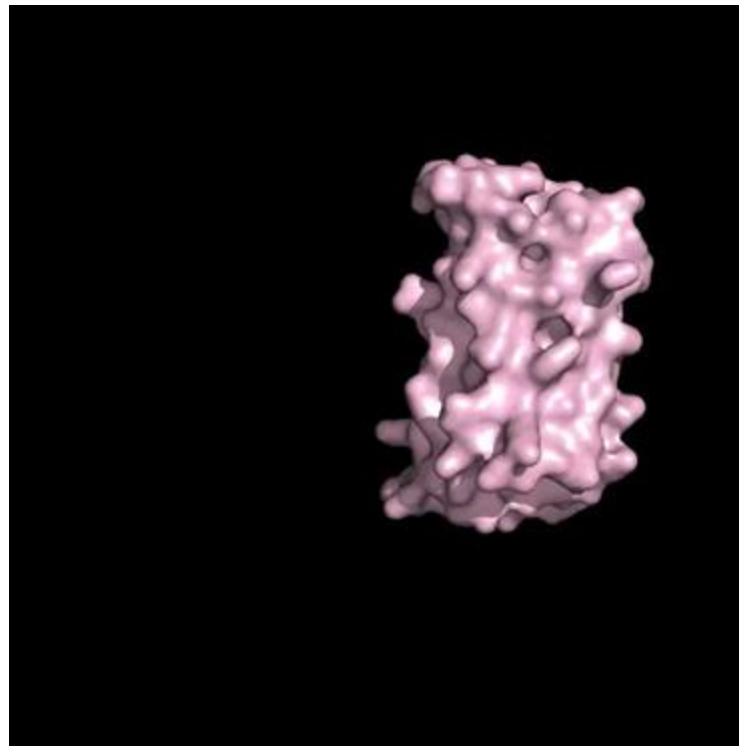


Next steps: Further improvements of the generative algorithm working together with the Genentech *AE team* for experimental testing

Generative AI for *de novo* design is the future!



Generative AI



Thank you!

Bonus slides - AlphaFold 3

Article

Improved protein structure prediction using potentials from deep learning

<https://doi.org/10.1038/s41586-019-1923-7>

Received: 2 April 2019

Accepted: 10 December 2019

Published online: 15 January 2020

Andrew W. Senior^{1,4*}, Richard Evans^{1,4}, John Jumper^{1,4}, James Kirkpatrick^{1,4}, Laurent Sifre^{1,4}, Tim Green¹, Chongli Qin¹, Augustin Židek¹, Alexander W. R. Nelson¹, Alex Bridgland¹, Hugo Penedones¹, Stig Petersen¹, Karen Simonyan¹, Steve Crossan¹, Pushmeet Kohli¹, David T. Jones^{1,3}, David Silver¹, Koray Kavukcuoglu² & Demis Hassabis¹

nature

Accelerated Article Preview

<https://doi.org/10.1038/s41586-024-03487-w>

Accurate structure prediction of biomolecular interactions with AlphaFold 3

Received: 19 December 2023

Accepted: 29 April 2024

Accelerated Article Preview

Cite this article as: Abramson, J. et al. Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature* <https://doi.org/10.1038/s41586-024-03487-w> (2024)

Josh Abramson, Jonas Adler, Jack Dunger, Richard Evans, Tim Green, Alexander Pritzel, Olaf Ronneberger, Lindsay Wilkins, Andrew J. Ballard, Joshua Baunberg, Sebastian W. Boderstein, David A. Evans, Chia-Chun Hung, Michael O'Neill, David Reiman, Kathryn Tunyasuvunakool, Zachary Wu, Abulá Zenguliyá, Krizia Arvaniti, Charles Beattie, Ottavia Bertoli, Alex Bridgland, Alexey Cherepanov, Miles Congreve, Alexander I. Cowen-Rivers, Andrew Cowie, Michael Figurnov, Fabian B. Fuchs, Hannah Gladman, Rishub Jain, Yousef A. Khan, Carlisle M. R. Lee, Kuba Perlin, Anne Potapenko, Pascal Sany, Ishdeep Singh, Adrian Stenula, Ashok Thillaisundaram, Catherine Tung, Sergei Yelisei, Ellen D. Zhong, Michael Zielinski, Augustin Židek, Victor Bajot, Pushmeet Kohli, Max Jaderberg, Demis Hassabis & John M. Jumper

2018

2020

2024

Article

Highly accurate protein structure prediction with AlphaFold

<https://doi.org/10.1038/s41586-021-03819-2>

Received: 11 May 2021

Accepted: 12 July 2021

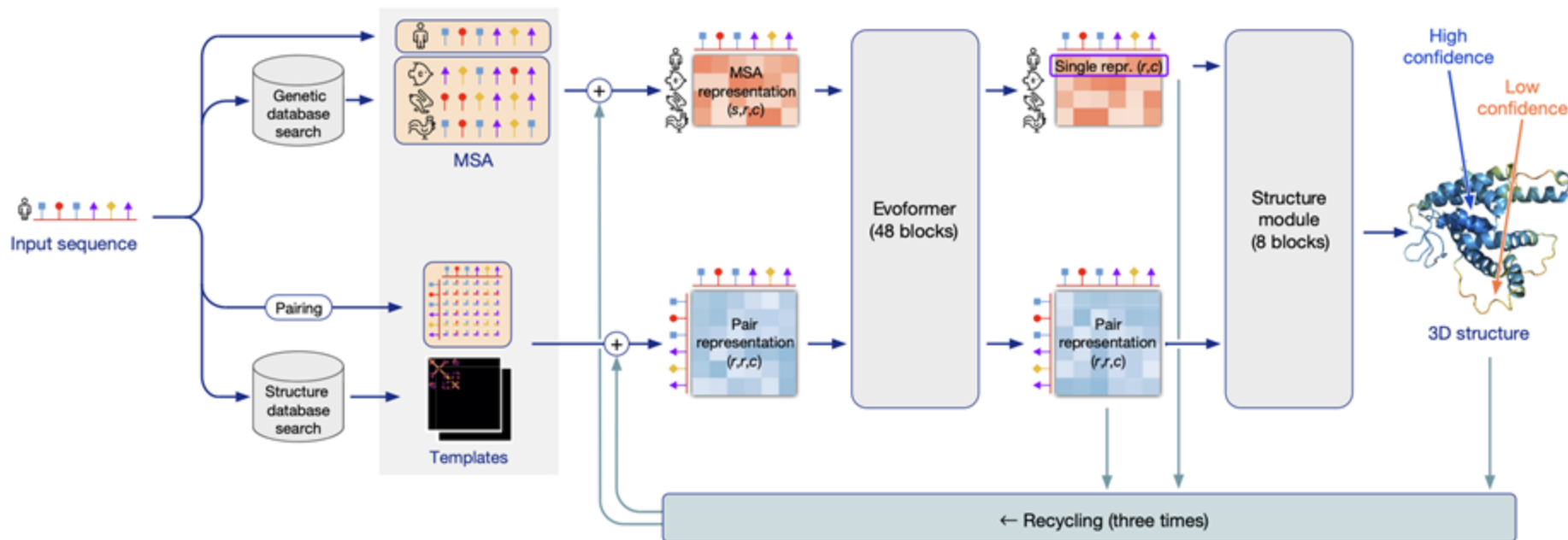
Published online: 15 July 2021

Open access

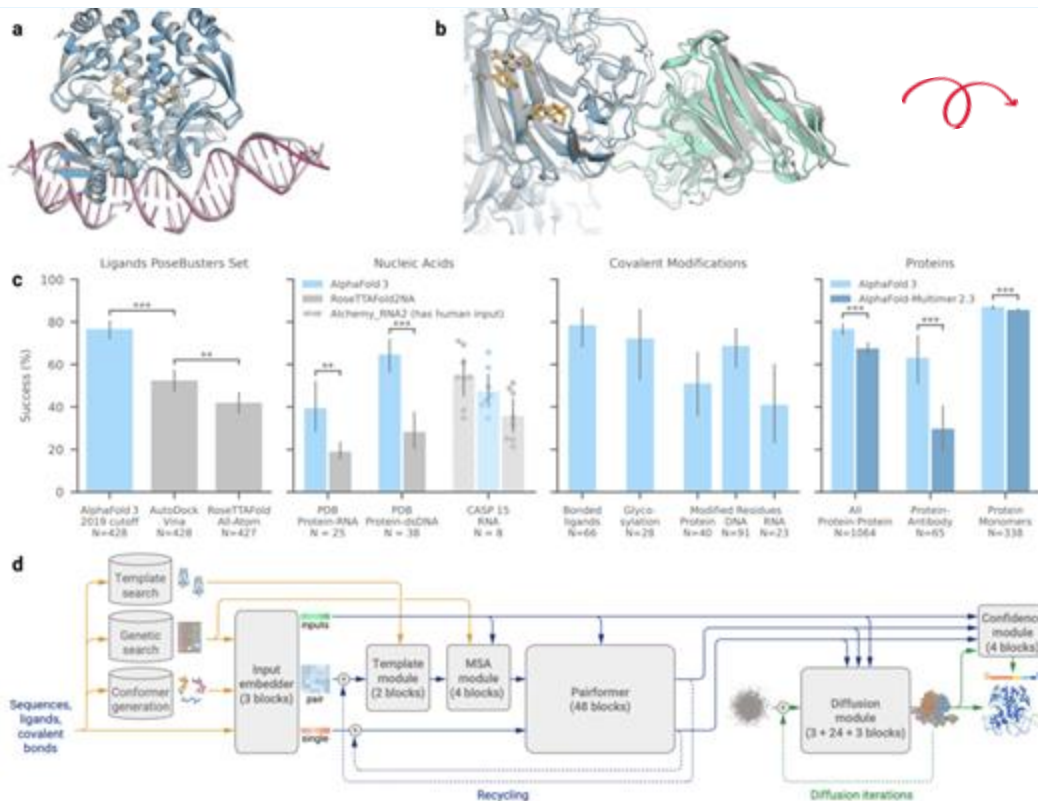
Check for updates

John Jumper^{1,4,10}, Richard Evans^{1,4}, Alexander Pritzel^{1,4}, Tim Green^{1,4}, Michael Figurnov^{1,4}, Olaf Ronneberger^{1,4}, Kathryn Tunyasuvunakool^{1,4}, Russ Bates^{1,4}, Augustin Židek^{1,4}, Anna Potapenko^{1,4}, Alex Bridgland^{1,4}, Clemens Meyer^{1,4}, Simon A. A. Kohl^{1,4}, Andrew J. Ballard^{1,4}, Andrew Cowie^{1,4}, Bernardino Romero-Paredes^{1,4}, Stanislav Nikolov^{1,4}, Rishub Jain^{1,4}, Jonas Adler^{1,4}, Trevor Back^{1,4}, Stig Petersen^{1,4}, David Reiman^{1,4}, Ellen Clancy^{1,4}, Michal Zielinski^{1,4}, Martin Steinegger^{1,4}, Michalina Pacholska^{1,4}, Tamas Berghammer^{1,4}, Sebastian Boderstein^{1,4}, David Silver^{1,4}, Oriol Vinyals^{1,4}, Andrew W. Senior^{1,4}, Koray Kavukcuoglu², Pushmeet Kohli² & Demis Hassabis^{1,4,10}

AlphaFold 3



AlphaFold 3



Increased structure prediction accuracy

Increased complex (binding) prediction accuracy

Generation is done all at the same time

Weisfeiler Leman graph isomorphism test

Algorithm 1: WL-1 algorithm (Weisfeiler & Lehmann, 1968)

Input: Initial node coloring $(h_1^{(0)}, h_2^{(0)}, \dots, h_N^{(0)})$

Output: Final node coloring $(h_1^{(T)}, h_2^{(T)}, \dots, h_N^{(T)})$

$t \leftarrow 0$;

repeat

for $v_i \in \mathcal{V}$ **do**

$h_i^{(t+1)} \leftarrow \text{hash} \left(\sum_{j \in \mathcal{N}_i} h_j^{(t)} \right)$;

$t \leftarrow t + 1$;

until *stable node coloring is reached*;
